

```

1 """
2 crypto_core.py – Cryptographic Primitive Wrappers
3 =====
4 TAD Reference: Section 3.1
5
6 Responsibility
7 -----
8 Thin, audited-library-backed wrappers around all raw cryptographic
9 primitives used by the DKE Messaging Engine.
10
11 Design Constraints (from TAD Section 3.1 & 5.3)
12 -----
13 * No hand-rolled ECC – X25519 operations delegate entirely to
14   ``cryptography.hazmat.primitives.asymmetric.x25519``.
15   Ed25519 signing/verification delegates entirely to
16   ``cryptography.hazmat.primitives.asymmetric.ed25519``.
17 * No hand-rolled HKDF – uses
18   ``cryptography.hazmat.primitives.kdf.hkdf.HKDF``.
19 * No disk writes – all key material lives in process memory only.
20 * The ``info`` parameter in ``hkdf_derive()`` MUST be a fixed,
21   context-specific ASCII string (e.g., ``"dke-session-init-v1"``) to
22   domain-separate different key usages.
23
24 Key Hierarchy (TAD Section 5.1)
25 -----
26 os.urandom()
27   ■
28   ■■■ Ed25519 identity keypair (long-term, one per process lifetime)
29   ■
30   ■ ■■■ sign(ephemeral ECDH pub) → 64-byte signature
31   ■
32   ■■■ X25519 ephemeral keypair (per-session)
33   ■
34   ■ ECDH(a_priv, b_pub) == ECDH(b_priv, a_pub)
35   ▼
36   Shared Secret (32 bytes raw)
37   ■
38   ■ HKDF-SHA256(secret, salt=None, info="dke-session-init-v1")
39   ▼
40   K0 – Initial Symmetric Key (32 bytes / AES-256)
41
42 MITM Mitigation (Section 3.1 – Identity Authentication)
43 -----
44 Before shared-secret derivation, each party signs its ephemeral X25519
45 public key with its long-term Ed25519 identity private key. The peer
46 verifies this signature against the known identity public key. An
47 attacker who substitutes a different ephemeral key cannot produce a
48 valid signature without the identity private key, so the forgery is
49 detected and the session is aborted.
50 """
51
52 from __future__ import annotations
53
54 from cryptography.exceptions import InvalidSignature # re-exported for callers
55 from cryptography.hazmat.primitives.asymmetric.ed25519 import (
56     Ed25519PrivateKey,
57     Ed25519PublicKey,
58 )
59 from cryptography.hazmat.primitives.asymmetric.x25519 import (
60     X25519PrivateKey,
61     X25519PublicKey,
62 )
63 from cryptography.hazmat.primitives.hashes import SHA256
64 from cryptography.hazmat.primitives.kdf.hkdf import HKDF
65
66
67 # -----

```

```

68 # Constants
69 # -----
70
71 #: Canonical domain-separation label for the initial session key.
72 #: Both parties MUST use the same string or key derivation will diverge.
73 SESSION_INFO: str = "dke-session-init-v1"
74
75 #: Size of a raw X25519 public key in bytes.
76 PUBLIC_KEY_SIZE: int = 32
77
78 #: Default output length for ``hkdf_derive()``, matching AES-256 key size.
79 DEFAULT_KEY_LENGTH: int = 32
80
81 #: Size of a raw Ed25519 public key in bytes.
82 IDENTITY_PUBLIC_KEY_SIZE: int = 32
83
84 #: Size of an Ed25519 signature in bytes.
85 SIGNATURE_SIZE: int = 64
86
87
88 # -----
89 # Public API
90 # -----
91
92
93 def generate_keypair() -> tuple[X25519PrivateKey, bytes]:
94     """Generate an X25519 key pair.
95
96     Delegates entirely to
97     ``cryptography.hazmat.primitives.asymmetric.x25519.X25519PrivateKey.generate()``.
98     No elliptic-curve arithmetic is performed in this module.
99
100     The private key object is kept in memory only and is never serialised
101     to disk by this function. The caller is responsible for ensuring the
102     private key is not persisted beyond the lifetime of the session.
103
104     Returns
105     -----
106     tuple[X25519PrivateKey, bytes]
107         A 2-tuple ``(private_key, public_key_bytes)`` where:
108
109         * ``private_key`` is the opaque ``X25519PrivateKey`` object
110           required by :func:`derive_shared_secret`.
111         * ``public_key_bytes`` is the raw 32-byte little-endian encoding
112           of the corresponding X25519 public key, ready to be sent to the
113           peer over the transport.
114
115     Examples
116     -----
117     >>> priv, pub = generate_keypair()
118     >>> assert len(pub) == 32
119
120     TAD Reference: Section 3.1 – generate_keypair()
121                   Section 5.4 – Algorithm Choices (X25519)
122     """
123     private_key: X25519PrivateKey = X25519PrivateKey.generate()
124     public_key_bytes: bytes = private_key.public_key().public_bytes_raw()
125     return private_key, public_key_bytes
126
127
128 def derive_shared_secret(
129     private_key: X25519PrivateKey,
130     peer_public_key_bytes: bytes,
131 ) -> bytes:
132     """Perform X25519 ECDH key agreement and return the raw shared secret.
133
134     Delegates entirely to the ``exchange()`` method of

```

```

135     ``cryptography.hazmat.primitives.asymmetric.x25519.X25519PrivateKey``.
136     No elliptic-curve arithmetic is performed in this module.
137
138     .. warning::
139         The returned 32 bytes are the raw Diffie-Hellman output. They
140         MUST be passed through :func:`hkdf_derive` before being used as an
141         AES-256 key. Using raw ECDH output directly as a cipher key is a
142         cryptographic error.
143
144     Parameters
145     -----
146     private_key : X25519PrivateKey
147         The local party's X25519 private key, as returned by the first
148         element of :func:`generate_keypair`.
149     peer_public_key_bytes : bytes
150         The peer's raw 32-byte X25519 public key received over the
151         transport. Must be exactly :data:`PUBLIC_KEY_SIZE` (32) bytes.
152
153     Returns
154     -----
155     bytes
156         32-byte raw Diffie-Hellman shared secret.
157
158     Raises
159     -----
160     ValueError
161         If ``peer_public_key_bytes`` is not exactly 32 bytes.
162     cryptography.exceptions.InvalidKey
163         If the peer's public key is not a valid X25519 point (e.g., the
164         low-order point attack).
165
166     Examples
167     -----
168     >>> priv_a, pub_a = generate_keypair()
169     >>> priv_b, pub_b = generate_keypair()
170     >>> secret_a = derive_shared_secret(priv_a, pub_b)
171     >>> secret_b = derive_shared_secret(priv_b, pub_a)
172     >>> assert secret_a == secret_b # ECDH symmetry
173
174     TAD Reference: Section 3.1 – derive_shared_secret()
175                     Section 5.1 – Key Hierarchy
176                     Section 5.2 – Security Properties
177     """
178     if len(peer_public_key_bytes) != PUBLIC_KEY_SIZE:
179         raise ValueError(
180             f"peer_public_key_bytes must be exactly {PUBLIC_KEY_SIZE} bytes, "
181             f"got {len(peer_public_key_bytes)}."
182         )
183
184     peer_public_key: X25519PublicKey = X25519PublicKey.from_public_bytes(
185         peer_public_key_bytes
186     )
187     shared_secret: bytes = private_key.exchange(peer_public_key)
188     return shared_secret
189
190
191 def hkdf_derive(
192     shared_secret: bytes,
193     info: str,
194     length: int = DEFAULT_KEY_LENGTH,
195 ) -> bytes:
196     """Derive a symmetric key from a raw shared secret using HKDF-SHA-256.
197
198     Delegates entirely to
199     ``cryptography.hazmat.primitives.kdf.hkdf.HKDF`` with:
200
201     * ``algorithm`` = SHA-256 (via ``cryptography.hazmat.primitives.hashes.SHA256``)

```

```

202 * ``salt`` = ``None`` (HKDF spec permits omitting salt; SHA-256 hash
203   length is used as the implicit salt per RFC 5869 §2.2)
204 * ``info`` = the UTF-8 encoding of the caller-supplied ``info`` string
205
206 No KDF arithmetic is performed in this module.
207
208 Parameters
209 -----
210 shared_secret : bytes
211     Raw output from :func:`derive_shared_secret`. Typically 32 bytes
212     for X25519, but HKDF-SHA-256 can accept any non-empty input.
213 info : str
214     Context / domain-separation label that binds the derived key to a
215     specific usage. Both parties MUST supply the identical ``info``
216     value or they will derive different keys.
217
218     Recommended value for the initial session key:
219     ``"dke-session-init-v1"`` (:data:`SESSION_INFO`).
220
221 length : int, optional
222     Number of output key bytes. Defaults to 32 (:data:`DEFAULT_KEY_LENGTH`),
223     matching the AES-256 key size. Must satisfy
224     ``1 <= length <= 255 * 32`` (HKDF-SHA-256 output limit).
225
226 Returns
227 -----
228 bytes
229     ``length``-byte derived symmetric key (K0 for the initial call).
230
231 Raises
232 -----
233 ValueError
234     If ``shared_secret`` is empty or ``length`` is out of range.
235 cryptography.exceptions.AlreadyFinalized
236     If the internal HKDF object is reused (it is always freshly
237     constructed here, so this should never occur).
238
239 Examples
240 -----
241 >>> priv_a, pub_a = generate_keypair()
242 >>> priv_b, pub_b = generate_keypair()
243 >>> secret = derive_shared_secret(priv_a, pub_b)
244 >>> k0 = hkdf_derive(secret, info=SESSION_INFO)
245 >>> assert len(k0) == 32
246
247 TAD Reference: Section 3.1 – hkdf_derive()
248                 Section 5.1 – Key Hierarchy
249                 Section 5.4 – Algorithm Choices (HKDF-SHA-256)
250 """
251 if not shared_secret:
252     raise ValueError("shared_secret must not be empty.")
253
254 hkdf = HKDF(
255     algorithm=SHA256(),
256     length=length,
257     salt=None, # RFC 5869 §2.2: omitted salt → hash-length zeros
258     info=info.encode("utf-8"),
259 )
260 derived_key: bytes = hkdf.derive(shared_secret)
261 return derived_key
262
263
264 # -----
265 # Ed25519 Identity Authentication
266 # -----
267
268

```

```

269 def generate_identity_keypair() -> tuple[Ed25519PrivateKey, bytes]:
270     """Generate a long-term Ed25519 identity key pair.
271
272     Delegates entirely to
273     ``cryptography.hazmat.primitives.asymmetric.ed25519.Ed25519PrivateKey.generate()``.
274     No elliptic-curve arithmetic is performed in this module.
275
276     The identity key pair is generated once per process lifetime and
277     never serialised to disk by this function. It is used only for
278     authenticating ephemeral X25519 public keys during session
279     establishment (MITM mitigation).
280
281     Returns
282     -----
283     tuple[Ed25519PrivateKey, bytes]
284         A 2-tuple ``(private_key, public_key_bytes)`` where:
285
286         * ``private_key`` is the opaque ``Ed25519PrivateKey`` object
287           required by :func:`sign_public_key`.
288         * ``public_key_bytes`` is the raw 32-byte encoding of the
289           corresponding Ed25519 public key, to be shared out-of-band
290           with the peer (the "identity fingerprint").
291
292     Examples
293     -----
294     >>> id_priv, id_pub = generate_identity_keypair()
295     >>> assert len(id_pub) == 32
296
297     TAD Reference: Section 3.1 – Identity Authentication (MITM Mitigation)
298     """
299     private_key: Ed25519PrivateKey = Ed25519PrivateKey.generate()
300     public_key_bytes: bytes = private_key.public_key().public_bytes_raw()
301     return private_key, public_key_bytes
302
303
304 def sign_public_key(
305     identity_private: Ed25519PrivateKey,
306     ecdh_pub_bytes: bytes,
307 ) -> bytes:
308     """Sign an ephemeral X25519 public key with the Ed25519 identity private key.
309
310     Produces a 64-byte Ed25519 signature over ``ecdh_pub_bytes``. The
311     peer will verify this signature against the sender's known identity
312     public key before accepting the ephemeral key for ECDH.
313
314     Delegates entirely to ``Ed25519PrivateKey.sign()``. No signing
315     arithmetic is performed in this module.
316
317     Parameters
318     -----
319     identity_private : Ed25519PrivateKey
320         The local party's long-term Ed25519 identity private key, as
321         returned by the first element of :func:`generate_identity_keypair`.
322     ecdh_pub_bytes : bytes
323         The raw 32-byte X25519 ephemeral public key to authenticate.
324         Must be exactly :data:`PUBLIC_KEY_SIZE` (32) bytes.
325
326     Returns
327     -----
328     bytes
329         64-byte Ed25519 signature.
330
331     Raises
332     -----
333     ValueError
334         If ``ecdh_pub_bytes`` is not exactly 32 bytes.
335

```

```

336 Examples
337 -----
338 >>> id_priv, id_pub = generate_identity_keypair()
339 >>> _, ecdh_pub = generate_keypair()
340 >>> sig = sign_public_key(id_priv, ecdh_pub)
341 >>> assert len(sig) == 64
342
343 TAD Reference: Section 3.1 – Identity Authentication (MITM Mitigation)
344 """
345 if len(ecdh_pub_bytes) != PUBLIC_KEY_SIZE:
346     raise ValueError(
347         f"ecdh_pub_bytes must be exactly {PUBLIC_KEY_SIZE} bytes, "
348         f"got {len(ecdh_pub_bytes)}."
349     )
350 return identity_private.sign(ecdh_pub_bytes)
351
352
353 def verify_public_key(
354     identity_pub_bytes: bytes,
355     ecdh_pub_bytes: bytes,
356     signature: bytes,
357 ) -> None:
358     """Verify an Ed25519 signature over an ephemeral X25519 public key.
359
360     Authenticates that ``ecdh_pub_bytes`` was signed by the holder of the
361     identity private key corresponding to ``identity_pub_bytes``. If
362     verification succeeds the function returns ``None``; if it fails, it
363     raises :class:`cryptography.exceptions.InvalidSignature`.
364
365     Delegates entirely to ``Ed25519PublicKey.verify()``. No verification
366     arithmetic is performed in this module.
367
368     .. warning::
369         This function MUST be called before :func:`derive_shared_secret`.
370         Proceeding with key derivation on an unauthenticated ephemeral key
371         defeats the MITM protection entirely.
372
373     Parameters
374     -----
375     identity_pub_bytes : bytes
376         The peer's raw 32-byte Ed25519 identity public key, obtained
377         out-of-band (e.g. printed at peer startup and manually compared –
378         similar to Signal's "safety numbers").
379         Must be exactly :data:`IDENTITY_PUBLIC_KEY_SIZE` (32) bytes.
380     ecdh_pub_bytes : bytes
381         The peer's raw 32-byte X25519 ephemeral public key received over
382         the transport.
383         Must be exactly :data:`PUBLIC_KEY_SIZE` (32) bytes.
384     signature : bytes
385         The 64-byte Ed25519 signature received alongside ``ecdh_pub_bytes``.
386         Must be exactly :data:`SIGNATURE_SIZE` (64) bytes.
387
388     Returns
389     -----
390     None
391         Returned only when the signature is valid.
392
393     Raises
394     -----
395     cryptography.exceptions.InvalidSignature
396         If the signature does not authenticate ``ecdh_pub_bytes`` under
397         ``identity_pub_bytes``. The caller MUST abort the session.
398     ValueError
399         If any argument has an unexpected length.
400
401     Examples
402     -----

```

```
403     >>> id_priv, id_pub = generate_identity_keypair()
404     >>> _, ecdh_pub = generate_keypair()
405     >>> sig = sign_public_key(id_priv, ecdh_pub)
406     >>> verify_public_key(id_pub, ecdh_pub, sig) # returns None – OK
407
408     TAD Reference: Section 3.1 – Identity Authentication (MITM Mitigation)
409     """
410     if len(identity_pub_bytes) != IDENTITY_PUBLIC_KEY_SIZE:
411         raise ValueError(
412             f"identity_pub_bytes must be exactly {IDENTITY_PUBLIC_KEY_SIZE} bytes, "
413             f"got {len(identity_pub_bytes)}."
414         )
415     if len(ecdh_pub_bytes) != PUBLIC_KEY_SIZE:
416         raise ValueError(
417             f"ecdh_pub_bytes must be exactly {PUBLIC_KEY_SIZE} bytes, "
418             f"got {len(ecdh_pub_bytes)}."
419         )
420     if len(signature) != SIGNATURE_SIZE:
421         raise ValueError(
422             f"signature must be exactly {SIGNATURE_SIZE} bytes, "
423             f"got {len(signature)}."
424         )
425
426     peer_pub_key: Ed25519PublicKey = Ed25519PublicKey.from_public_bytes(identity_pub_bytes)
427     # Raises cryptography.exceptions.InvalidSignature on failure.
428     peer_pub_key.verify(signature, ecdh_pub_bytes)
```

```

1  """
2  dke_engine.py – Dynamic Key Evolution State Machine
3  =====
4  TAD Reference: Section 3.2
5
6  Responsibility
7  -----
8  Maintains per-session ratchet state and advances the symmetric key after
9  every successful send or receive, providing **forward secrecy** within a
10 session.
11
12 Key Evolution Formula (TAD Section 3.2)
13 -----
14  $K_{i+1} = \text{SHA-256}(K_i || \text{nonce}_i || \text{seq}_i)$ 
15
16 where `||` is concatenation and `seq_i` is encoded as a fixed-width
17 big-endian unsigned 64-bit integer.
18
19 Design: Two Independent Ratchet Chains
20 -----
21 Each session maintains **two completely independent** key chains:
22
23 Outbound chain (local send → peer receive)
24   outbound.current_key – current AES-256 encryption key
25   outbound.sequence_number – next sequence number to embed in envelope
26   outbound.last_nonce – nonce used in the most recent send step
27
28 Inbound chain (peer send → local receive)
29   inbound.current_key – current AES-256 decryption key
30   inbound.sequence_number – next sequence number expected from peer
31   inbound.last_nonce – nonce used in the most recent receive step
32
33 Both chains are seeded from the same K0 at session start, then diverge
34 independently. This means:
35
36 * A long run of outbound-only messages does NOT advance the inbound key
37   (no key desynchronisation risk).
38 * The outbound and inbound chains cannot interfere with each other.
39 * The cross-matching invariant always holds after any exchange:
40
41     sender.outbound.current_key == receiver.inbound.current_key
42
43 Memory Safety Rule (TAD Section 3.2)
44 -----
45 After `advance_key()` produces `K_{i+1}`, the engine overwrites the
46 memory location of `K_i` with zeros before releasing the reference.
47 `current_key` is stored as a `bytearray` so this zeroing is in-place.
48
49 Sequence Number Policy (TAD Section 3.2)
50 -----
51 * Outbound and inbound sequence numbers advance independently.
52 * Sequence numbers are STRICTLY MONOTONIC – no gaps, no reuse.
53
54 Ratchet Lifecycle (one direction shown)
55 -----
56 K0 (from hkdf_derive)
57 ■
58 ■■■ send / receive message 0 (nonce_0)
59 ▼
60 K1 = SHA-256(K0 || nonce_0 || 0) ← K0 memory overwritten
61 ■
62 ■■■ send / receive message 1 (nonce_1)
63 ▼
64 K2 = SHA-256(K1 || nonce_1 || 1) ← K1 memory overwritten
65 ■
66 ...
67

```

```

68 Typical usage in main_chat.py
69 -----
70 Send path::
71
72     key, seq = dke_engine.outbound_params(state)
73     nonce    = os.urandom(12)
74     ct, tag  = protocol.encrypt(key, plaintext, nonce)
75     envelope = protocol.encode_envelope(seq, nonce, ct, tag)
76     transport.send(envelope)
77     dke_engine.rotate_send(state, nonce)      # erases old outbound key
78
79 Receive path::
80
81     seq_r, nonce_r, ct_r, tag_r = protocol.decode_envelope(raw)
82     key_r    = dke_engine.inbound_key(state, seq_r) # validates seq
83     plaintext = protocol.decrypt(key_r, ct_r, nonce_r, tag_r) # raises on fail
84     dke_engine.rotate_receive(state, nonce_r, seq_r) # erases old inbound key
85 """
86
87 from __future__ import annotations
88
89 import hashlib
90 from dataclasses import dataclass, field
91
92 # -----
93 # Constants
94 # -----
95
96 #: Expected key size in bytes (AES-256).
97 KEY_SIZE: int = 32
98
99 #: Expected nonce size in bytes (AES-GCM).
100 NONCE_SIZE: int = 12
101
102 #: Width (bytes) used to encode a sequence number in the hash input.
103 #: uint64 big-endian → 8 bytes.
104 _SEQ_WIDTH: int = 8
105
106
107 # -----
108 # Exceptions
109 # -----
110
111
112 class SequenceError(ValueError):
113     """Raised when a received sequence number does not match the expected value.
114
115     This indicates either an out-of-order delivery or a replay attempt.
116     The session continues; only the individual message is dropped.
117
118     TAD Reference: Section 7 – Failure Modes (Out-of-order / Replay)
119     """
120
121
122 # -----
123 # Direction State
124 # -----
125
126
127 @dataclass
128 class DirectionState:
129     """Ratchet state for one message direction (outbound or inbound).
130
131     Holds the current symmetric key, the next expected / next-to-use
132     sequence number, and the nonce used in the most recent ratchet step
133     (kept for audit / logging only).
134

```

```

135     ``current_key`` is a :class:`bytearray` so it can be overwritten
136     in-place with zeros before the reference is released (best-effort
137     memory erasure, TAD Section 3.2).
138
139     Attributes
140     -----
141     current_key : bytearray
142         The current 32-byte AES-256 symmetric key for this direction.
143     sequence_number : int
144         For the **outbound** direction: the sequence number that will be
145         embedded in the *next* outgoing envelope.
146         For the **inbound** direction: the sequence number *expected* in
147         the next incoming envelope.
148         Starts at 0; strictly monotonically increasing.
149     last_nonce : bytes
150         The 12-byte nonce from the most recent ratchet step in this
151         direction. Initialised to 12 zero bytes; used for audit only.
152
153     TAD Reference: Section 3.2 – Session State (per-direction variant)
154     """
155
156     current_key: bytearray
157     sequence_number: int = 0
158     last_nonce: bytes = field(default_factory=lambda: b"\x00" * NONCE_SIZE)
159
160     def wipe(self) -> None:
161         """Best-effort erasure of current key material in memory."""
162         _erase(self.current_key)
163
164
165 # -----
166 # Session State
167 # -----
168
169
170 @dataclass
171 class DKESessionState:
172     """Full DKE session state with **separate** outbound and inbound chains.
173
174     Both chains are seeded from the same K0 (from ``crypto_core.hkdf_derive``)
175     and then evolve completely independently. This guarantees:
176
177     * Sending many messages does not advance the receive-side key.
178     * The cross-matching invariant always holds after any exchange:
179
180         sender.outbound.current_key == receiver.inbound.current_key
181
182     Attributes
183     -----
184     outbound : DirectionState
185         Ratchet chain for messages *sent* by this party.
186     inbound : DirectionState
187         Ratchet chain for messages *received* by this party.
188
189     TAD Reference: Section 3.2 – Session State
190                   Section 3.2 – Sequence Number Policy
191     """
192
193     outbound: DirectionState
194     inbound: DirectionState
195
196     def wipe(self) -> None:
197         """Best-effort erasure of both outbound and inbound key material."""
198         self.outbound.wipe()
199         self.inbound.wipe()
200
201 # -----

```

```

202 # Constructor helper
203 # -----
204
205 @classmethod
206 def from_k0(cls, k0: bytes) -> DKESessionState:
207     """Create a fresh session state seeded from the initial derived key K0.
208
209     Both the outbound and inbound chains start from the same K0.
210     They diverge independently as messages are exchanged.
211
212     Parameters
213     -----
214     k0 : bytes
215         32-byte initial symmetric key produced by
216         ``crypto_core.hkdf_derive()``. Must be exactly
217         :data:`KEY_SIZE` (32) bytes.
218
219     Returns
220     -----
221     DKESessionState
222         Fresh session state with both sequence counters at 0 and both
223         chain keys set to mutable copies of ``k0``.
224
225     Raises
226     -----
227     ValueError
228         If ``k0`` is not exactly 32 bytes.
229
230     Examples
231     -----
232     >>> from crypto_core import generate_keypair, derive_shared_secret, hkdf_derive, SESSION_INFO
233     >>> priv_a, pub_a = generate_keypair()
234     >>> priv_b, pub_b = generate_keypair()
235     >>> secret = derive_shared_secret(priv_a, pub_b)
236     >>> k0 = hkdf_derive(secret, info=SESSION_INFO)
237     >>> state = DKESessionState.from_k0(k0)
238     >>> assert len(state.outbound.current_key) == 32
239     >>> assert len(state.inbound.current_key) == 32
240
241     TAD Reference: Section 6.1 – Session Establishment
242     """
243     if len(k0) != KEY_SIZE:
244         raise ValueError(
245             f"k0 must be exactly {KEY_SIZE} bytes, got {len(k0)}."
246         )
247     return cls(
248         outbound=DirectionState(current_key=bytearray(k0)),
249         inbound=DirectionState(current_key=bytearray(k0)),
250     )
251
252
253 # -----
254 # Private helpers
255 # -----
256
257
258 def _erase(buf: bytearray) -> None:
259     """Best-effort in-place zeroing of a ``bytearray`` key buffer.
260
261     Overwrites every byte position with ``0x00``. Python's object model
262     (reference counting, interning, GC) means true cryptographic erasure
263     is not possible at the language level; this is the best effort available.
264
265     Parameters
266     -----
267     buf : bytearray
268         The mutable key buffer to zero.

```

```

269
270     TAD Reference: Section 3.2 – Memory Safety Rule
271     """
272     for i in range(len(buf)):
273         buf[i] = 0
274
275
276 # -----
277 # Core ratchet – pure function
278 # -----
279
280
281 def advance_key(
282     current_key: bytes | bytearray,
283     nonce: bytes,
284     sequence: int,
285 ) -> bytes:
286     """Advance the ratchet and return the next symmetric key.
287
288     Computes the one-way hash chain step::
289
290         next_key = SHA-256(
291             current_key                # 32 bytes
292             || nonce                    # 12 bytes
293             || sequence.to_bytes(8, 'big') # 8 bytes
294         )
295
296     This is a pure function – it does not modify ``current_key`` or
297     any session state. The caller is responsible for:
298
299     1. Zeroing the old key buffer (use :func:`rotate_send` or
300        :func:`rotate_receive`, which handle this automatically).
301     2. Storing the returned bytes as the new ``DirectionState.current_key``.
302     3. Incrementing the appropriate ``sequence_number``.
303
304     The SHA-256 one-way property ensures ``current_key`` (Ki) cannot be
305     recovered from ``next_key`` (Ki+1), providing forward secrecy
306     within a session (TAD Section 5.2).
307
308     Parameters
309     -----
310     current_key : bytes or bytearray
311         The current 32-byte AES-256 key (Ki) for this direction.
312     nonce : bytes
313         The 12-byte AES-GCM nonce associated with the message that
314         triggers this advance. Including the nonce makes each step
315         unique even across session restarts.
316     sequence : int
317         The sequence number of the message that triggers this advance.
318         Encoded as a big-endian unsigned 64-bit integer before hashing.
319         Must satisfy ``0 <= sequence < 2**64``.
320
321     Returns
322     -----
323     bytes
324         32-byte next symmetric key (Ki+1).
325
326     Raises
327     -----
328     ValueError
329         If ``current_key`` is not exactly :data:`KEY_SIZE` bytes, ``nonce``
330         is not exactly :data:`NONCE_SIZE` bytes, or ``sequence`` is negative.
331
332     Examples
333     -----
334     >>> import os
335     >>> k0 = os.urandom(32)

```

```

336     >>> nonce = os.urandom(12)
337     >>> k1 = advance_key(k0, nonce, sequence=0)
338     >>> assert len(k1) == 32
339     >>> assert k1 != k0 # key changed
340     >>> assert advance_key(k0, nonce, 0) == k1 # deterministic
341
342     TAD Reference: Section 3.2 – advance_key() & Key Evolution Formula
343                   Section 5.2 – Forward Secrecy, Key Non-reversibility
344     """
345     if len(current_key) != KEY_SIZE:
346         raise ValueError(
347             f"current_key must be {KEY_SIZE} bytes, got {len(current_key)}."
348         )
349     if len(nonce) != NONCE_SIZE:
350         raise ValueError(
351             f"nonce must be {NONCE_SIZE} bytes, got {len(nonce)}."
352         )
353     if sequence < 0:
354         raise ValueError(
355             f"sequence must be non-negative, got {sequence}."
356         )
357
358     seq_bytes: bytes = sequence.to_bytes(_SEQ_WIDTH, byteorder="big")
359     hash_input: bytes = bytes(current_key) + nonce + seq_bytes
360     return hashlib.sha256(hash_input).digest()
361
362
363 # -----
364 # Sequence validation
365 # -----
366
367
368 def validate_sequence(expected: int, received: int) -> None:
369     """Assert that a received sequence number exactly matches the expected value.
370
371     The inbound chain maintains a strict counter of how many messages
372     have been processed from the peer. Any deviation – caused by loss,
373     reordering, or an adversarial replay – is treated as a fatal error for
374     that individual message. The session itself continues (TAD Section 7).
375
376     Parameters
377     -----
378     expected : int
379         The next sequence number the receiver anticipates, i.e., the
380         current value of ``DirectionState.sequence_number`` on the inbound
381         chain.
382     received : int
383         The sequence number extracted from the decoded envelope.
384
385     Raises
386     -----
387     SequenceError
388         If ``received != expected``.
389
390     Examples
391     -----
392     >>> validate_sequence(expected=3, received=3) # OK – no exception
393     >>> validate_sequence(expected=3, received=4) # raises SequenceError
394
395     TAD Reference: Section 3.2 – Sequence Number Policy
396                   Section 7 – Failure Modes (Out-of-order / Replay)
397     """
398     if received != expected:
399         raise SequenceError(
400             f"Sequence mismatch: expected {expected}, received {received}. "
401             "Message dropped (possible replay or out-of-order delivery)."
402         )

```

```

403
404
405 # -----
406 # High-level session helpers (called by main_chat.py)
407 # -----
408
409
410 def outbound_params(state: DKESessionState) -> tuple[bytes, int]:
411     """Return a key snapshot and sequence number for the next outbound message.
412
413     Call this immediately before ``protocol.encrypt()`` to obtain the key
414     and sequence number for the current outgoing message. Returns an
415     immutable ``bytes`` snapshot – the outbound chain is not advanced.
416     Call :func:`rotate_send` after a successful send to advance the chain
417     and erase the old key.
418
419     Parameters
420     -----
421     state : DKESessionState
422         Active session state.
423
424     Returns
425     -----
426     tuple[bytes, int]
427         ``(key_snapshot, outbound_seq)`` where:
428
429         * ``key_snapshot`` – immutable 32-byte copy of
430           ``state.outbound.current_key`` for use in ``protocol.encrypt()``.
431         * ``outbound_seq`` – the sequence number to embed in the envelope
432           (``protocol.encode_envelope()`` argument).
433
434     TAD Reference: Section 6.2 – Steady-State Send
435     """
436     return bytes(state.outbound.current_key), state.outbound.sequence_number
437
438
439 def inbound_key(state: DKESessionState, received_seq: int) -> bytes:
440     """Validate the received sequence number and return the inbound key snapshot.
441
442     Call this immediately before ``protocol.decrypt()`` to get the key
443     that should authenticate and decrypt the incoming message. Raises
444     :class:`SequenceError` before touching or returning the key if the
445     sequence number is wrong, so ratchet state is never modified on a bad
446     message.
447
448     Call :func:`rotate_receive` only after a successful
449     ``protocol.decrypt()`` returns (authentication tag verified).
450
451     Parameters
452     -----
453     state : DKESessionState
454         Active session state.
455     received_seq : int
456         Sequence number extracted from the decoded envelope.
457
458     Returns
459     -----
460     bytes
461         Immutable 32-byte copy of ``state.inbound.current_key`` for use in
462         ``protocol.decrypt()``.
463
464     Raises
465     -----
466     SequenceError
467         If ``received_seq`` does not equal the expected inbound sequence
468         counter. State is NOT modified when this is raised.
469

```

```

470     TAD Reference: Section 3.2 – Sequence Number Policy
471                     Section 7 – Failure Modes (Out-of-order / Replay)
472                     Section 6.2 – Steady-State Receive
473     """
474     validate_sequence(state.inbound.sequence_number, received_seq)
475     return bytes(state.inbound.current_key)
476
477 def rotate_send(state: DKESessionState, nonce: bytes) -> None:
478     """Advance the **outbound** ratchet after a successful send.
479
480     Steps performed atomically from the caller's perspective:
481
482     1. Read ``state.outbound.sequence_number`` (the seq of the just-sent message).
483     2. Compute ``K_(i+1) = advance_key(outbound.current_key, nonce, seq)``.
484     3. Zero ``state.outbound.current_key`` buffer in-place (best-effort erasure).
485     4. Replace ``state.outbound.current_key`` with ``bytearray(K_(i+1))``.
486     5. Update ``state.outbound.last_nonce``.
487     6. Increment ``state.outbound.sequence_number``.
488
489     The **inbound** chain is not touched.
490
491     .. warning::
492         Call this **only after** the envelope has been successfully sent
493         over the transport. If the send fails, leave state unchanged so
494         the same key and sequence number can be used for a retry.
495
496     Parameters
497     -----
498     state : DKESessionState
499         Active session state (``state.outbound`` mutated in-place).
500     nonce : bytes
501         The 12-byte nonce that was passed to ``protocol.encrypt()`` for
502         this message.
503
504     Raises
505     -----
506     ValueError
507         If ``nonce`` is not exactly 12 bytes (propagated from
508         :func:`advance_key`).
509
510     TAD Reference: Section 3.2 – Memory Safety Rule & Key Evolution Formula
511                     Section 6.2 – Steady-State Send
512     """
513     seq: int = state.outbound.sequence_number
514     next_key: bytes = advance_key(state.outbound.current_key, nonce, seq)
515
516     _erase(state.outbound.current_key) # best-effort erasure
517     state.outbound.current_key = bytearray(next_key)
518     state.outbound.last_nonce = nonce
519     state.outbound.sequence_number += 1
520
521
522 def rotate_receive(
523     state: DKESessionState,
524     nonce: bytes,
525     received_seq: int,
526 ) -> None:
527     """Advance the **inbound** ratchet after a successful receive and decrypt.
528
529     Steps performed:
530
531     1. Validate ``received_seq`` against the expected inbound counter –
532        raises :class:`SequenceError` if wrong; state is **not** modified.
533     2. Compute ``K_(i+1) = advance_key(inbound.current_key, nonce, received_seq)``.
534     3. Zero ``state.inbound.current_key`` buffer in-place (best-effort erasure).
535     4. Replace ``state.inbound.current_key`` with ``bytearray(K_(i+1))``.

```

```

537     5. Update ``state.inbound.last_nonce``.
538     6. Increment ``state.inbound.sequence_number``.
539
540     The outbound chain is not touched.
541
542     .. warning::
543         Call this only after ``protocol.decrypt()`` has returned
544         successfully (authentication tag verified). If decryption raises,
545         do NOT call this function – the old key must remain so the next
546         valid message can be processed.
547
548     Parameters
549     -----
550     state : DKESessionState
551         Active session state (``state.inbound`` mutated in-place).
552     nonce : bytes
553         The 12-byte nonce extracted from the decoded envelope.
554     received_seq : int
555         The sequence number extracted from the decoded envelope.
556
557     Raises
558     -----
559     SequenceError
560         If ``received_seq`` does not match the expected inbound counter.
561         State is NOT modified when this is raised.
562     ValueError
563         If ``nonce`` is not exactly 12 bytes (propagated from
564         :func:`advance_key`).
565
566     TAD Reference: Section 3.2 – Memory Safety Rule & Key Evolution Formula
567                   Section 7 – Failure Modes (Out-of-order / Replay)
568                   Section 6.2 – Steady-State Receive
569
570     """
571     validate_sequence(state.inbound.sequence_number, received_seq) # raises before touching state
572
573     if state.inbound.sequence_number > 0 and nonce == state.inbound.last_nonce:
574         raise ValueError("Nonce reuse detected: incoming message reused last_nonce.")
575
576     next_key: bytes = advance_key(state.inbound.current_key, nonce, received_seq)
577
578     _erase(state.inbound.current_key) # best-effort erasure
579     state.inbound.current_key = bytearray(next_key)
580     state.inbound.last_nonce = nonce
581     state.inbound.sequence_number += 1

```

```

1 """
2 protocol.py – Envelope Encoding & AEAD Wrappers
3 =====
4 TAD Reference: Section 3.3, Section 4
5
6 Responsibility
7 -----
8 * Serialise and deserialise binary message envelopes (TAD Section 4.1).
9 * Wrap AES-256-GCM encryption and decryption from the ``cryptography``
10   (pyca) library.
11
12 This module does NOT implement AES or GCM itself.
13
14 Wire Format (TAD Section 4.1)
15 -----
16 All multi-byte integer fields are BIG-ENDIAN.
17
18   Byte offset  Field                Size    struct token
19   -----
20   0            Magic (b"DKE1")       4 B      4s
21   4            Version                1 B      B
22   5            Flags                  1 B      B (reserved; must be 0x00 in v1)
23   6            Sequence Number        8 B      Q (uint64, big-endian)
24   14           Nonce                  12 B     12s (AES-GCM nonce)
25   26           Ciphertext Length      4 B      I (uint32, big-endian)
26   30           Ciphertext              N B      (variable)
27   30+N         Authentication Tag     16 B     (AES-GCM 128-bit tag)
28
29 Fixed header = 30 bytes (packed with ``_HEADER_STRUCT``).
30 Minimum envelope size = 46 bytes (header + empty ciphertext + 16-byte tag).
31
32 ``decode_envelope()`` validates:
33   1. Total length >= 46 bytes (before reading any field).
34   2. Magic == b"DKE1".
35   3. Version == 0x01.
36   4. Flags == 0x00 (reserved in v1).
37   5. Total length == 30 + ct_len + 16 (declared vs. actual consistency).
38
39 AEAD Contract (TAD Section 3.3)
40 -----
41 * ``encrypt()`` delegates to
42   ``cryptography.hazmat.primitives.ciphers.aead.AESGCM``.
43 * AESGCM.encrypt() returns ``ciphertext || tag`` concatenated; this module
44   splits them before returning so the caller handles ciphertext and tag as
45   separate fields (matching the envelope wire format).
46 * ``decrypt()`` re-concatenates ``ciphertext + tag`` before passing to
47   AESGCM.decrypt(), which raises ``cryptography.exceptions.InvalidTag`` on
48   any authentication failure.
49 * ``decrypt()`` NEVER returns plaintext on failure (fail-closed, TAD §7.1).
50 """
51
52 from __future__ import annotations
53
54 import struct
55
56 from cryptography.exceptions import InvalidTag # re-exported for callers
57 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
58
59 # -----
60 # Wire-format constants
61 # -----
62
63 #: 4-byte protocol identifier at the start of every envelope.
64 MAGIC: bytes = b"DKE1"
65
66 #: Current protocol version byte.
67 VERSION: int = 0x01

```

```

68
69 #: Size of the AES-GCM authentication tag in bytes.
70 TAG_SIZE: int = 16
71
72 #: Size of the AES-GCM nonce in bytes.
73 NONCE_SIZE: int = 12
74
75 #: Size of the AES-256 key in bytes.
76 KEY_SIZE: int = 32
77
78 #: Minimum legal envelope size in bytes (TAD Section 4.2).
79 #: = 4 (magic) + 1 (version) + 1 (flags) + 8 (seq) + 12 (nonce)
80 #:   + 4 (ct_len) + 0 (empty ct) + 16 (tag)
81 MIN_ENVELOPE_SIZE: int = 46
82
83 # -----
84 # Struct for the fixed-size envelope header (30 bytes)
85 # -----
86 # Format characters (all big-endian via '!'):
87 # 4s  - 4-byte magic string
88 # B   - version (uint8)
89 # B   - flags (uint8)
90 # Q   - sequence number (uint64)
91 # 12s - nonce (12 raw bytes)
92 # I   - ciphertext length (uint32)
93 _HEADER_STRUCT: struct.Struct = struct.Struct("!4sBBQ12sI")
94
95 #: Byte length of the fixed header portion (everything before ciphertext).
96 HEADER_SIZE: int = _HEADER_STRUCT.size # == 30
97
98 assert HEADER_SIZE == 30, "Header struct size mismatch - check format string."
99 assert MIN_ENVELOPE_SIZE == HEADER_SIZE + TAG_SIZE, (
100     "MIN_ENVELOPE_SIZE must equal HEADER_SIZE + TAG_SIZE."
101 )
102
103 # -----
104 # Exceptions
105 # -----
106
107
108 class ParseError(Exception):
109     """Raised by :func:`decode_envelope` when the envelope is malformed.
110
111     Covers all structural problems: wrong length, bad magic, unsupported
112     version, non-zero reserved flags, or an inconsistent ciphertext-length
113     field.
114
115     The session continues after a ``ParseError``; only the individual
116     message is dropped.
117
118     TAD Reference: Section 7 - Failure Modes (Malformed envelope)
119     """
120
121
122 # Re-export InvalidTag so callers can catch it without importing from
123 # cryptography directly.
124 __all__ = [
125     "MAGIC", "VERSION", "TAG_SIZE", "NONCE_SIZE", "KEY_SIZE",
126     "MIN_ENVELOPE_SIZE", "HEADER_SIZE",
127     "ParseError", "InvalidTag",
128     "encode_envelope", "decode_envelope",
129     "encrypt", "decrypt",
130 ]
131
132
133 # -----
134 # Envelope encoding

```

```

135 # -----
136
137
138 def encode_envelope(
139     sequence: int,
140     nonce: bytes,
141     ciphertext: bytes,
142     tag: bytes,
143     flags: int = 0x00,
144 ) -> bytes:
145     """Serialise message fields into the DKE binary wire format.
146
147     Packs the fixed header using :data:`_HEADER_STRUCT` (big-endian) then
148     appends the variable-length ciphertext and the 16-byte authentication
149     tag.
150
151     Parameters
152     -----
153     sequence : int
154         Unsigned 64-bit sequence number for this message direction.
155         Must satisfy ``0 <= sequence < 2**64``.
156     nonce : bytes
157         12-byte AES-GCM nonce for this message (from ``os.urandom(12)``).
158         Must be exactly :data:`NONCE_SIZE` bytes.
159     ciphertext : bytes
160         AES-256-GCM ciphertext returned by :func:`encrypt`.
161         May be empty (zero-length plaintext is valid).
162     tag : bytes
163         16-byte AES-256-GCM authentication tag returned by :func:`encrypt`.
164         Must be exactly :data:`TAG_SIZE` bytes.
165     flags : int, optional
166         Reserved flags byte. Must be ``0x00`` in protocol version 1.
167         Defaults to ``0x00``.
168
169     Returns
170     -----
171     bytes
172         Fully framed binary envelope (>= :data:`MIN_ENVELOPE_SIZE` bytes),
173         ready for transmission over the transport.
174
175     Raises
176     -----
177     ValueError
178         If ``nonce`` is not 12 bytes, ``tag`` is not 16 bytes, or
179         ``sequence`` is out of the uint64 range.
180
181     Examples
182     -----
183     >>> import os
184     >>> key = os.urandom(32)
185     >>> nonce = os.urandom(12)
186     >>> ct, tag = encrypt(key, b"hello", nonce)
187     >>> envelope = encode_envelope(sequence=0, nonce=nonce, ciphertext=ct, tag=tag)
188     >>> assert len(envelope) >= 46
189
190     TAD Reference: Section 3.3 – encode_envelope()
191                     Section 4.1 – Envelope Structure
192     """
193     if len(nonce) != NONCE_SIZE:
194         raise ValueError(
195             f"nonce must be {NONCE_SIZE} bytes, got {len(nonce)}."
196         )
197     if len(tag) != TAG_SIZE:
198         raise ValueError(
199             f"tag must be {TAG_SIZE} bytes, got {len(tag)}."
200         )
201     if not (0 <= sequence < 2**64):

```

```

202         raise ValueError(
203             f"sequence must be in [0, 2^64), got {sequence}."
204         )
205
206     header: bytes = _HEADER_STRUCT.pack(
207         MAGIC,
208         VERSION,
209         flags,
210         sequence,
211         nonce,
212         len(ciphertext),
213     )
214     return header + ciphertext + tag
215
216
217 # -----
218 # Envelope decoding
219 # -----
220
221
222 def decode_envelope(raw: bytes) -> tuple[int, bytes, bytes, bytes]:
223     """Parse and validate a raw binary envelope.
224
225     Performs **strict length checks before parsing any fields** to avoid
226     information leakage from partial parsing of malformed input.
227
228     Validation order
229     -----
230     1. ``len(raw) >= 46`` – absolute minimum check (no field reads before this).
231     2. Unpack fixed header via :data:`_HEADER_STRUCT`.
232     3. ``magic == b"DKE1"`` – protocol identifier.
233     4. ``version == 0x01`` – supported version.
234     5. ``flags == 0x00`` – reserved bits (v1 must be zero).
235     6. ``len(raw) == HEADER_SIZE + ct_len + TAG_SIZE`` – declared vs. actual
236         consistency. Rejects both truncated and padded envelopes.
237
238     Parameters
239     -----
240     raw : bytes
241         Raw bytes received from the transport.
242
243     Returns
244     -----
245     tuple[int, bytes, bytes, bytes]
246         ``(sequence, nonce, ciphertext, tag)`` where:
247
248         * ``sequence`` – uint64 sequence number.
249         * ``nonce`` – 12-byte AES-GCM nonce.
250         * ``ciphertext`` – variable-length ciphertext bytes.
251         * ``tag`` – 16-byte AES-GCM authentication tag.
252
253     Raises
254     -----
255     ParseError
256         On any structural problem with the envelope. Plaintext is never
257         touched; the failure is purely structural.
258
259     Examples
260     -----
261     >>> import os
262     >>> key = os.urandom(32)
263     >>> nonce = os.urandom(12)
264     >>> ct, tag = encrypt(key, b"hello", nonce)
265     >>> envelope = encode_envelope(0, nonce, ct, tag)
266     >>> seq, n, c, t = decode_envelope(envelope)
267     >>> assert seq == 0 and n == nonce and c == ct and t == tag
268

```

```

269     TAD Reference: Section 3.3 – decode_envelope()
270         Section 4.1 – Envelope Structure
271         Section 4.2 – Minimum Envelope Size
272         Section 7 – Failure Modes (Malformed envelope)
273     """
274     # -----
275     # Check 1: absolute minimum length (no struct reads before this)
276     # -----
277     if len(raw) < MIN_ENVELOPE_SIZE:
278         raise ParseError(
279             f"Envelope too short: got {len(raw)} bytes, "
280             f"minimum is {MIN_ENVELOPE_SIZE}."
281         )
282
283     # -----
284     # Check 2: unpack fixed header
285     # -----
286     try:
287         magic, version, flags, sequence, nonce, ct_len = (
288             _HEADER_STRUCT.unpack_from(raw, offset=0)
289         )
290     except struct.error as exc:
291         raise ParseError(f"Header unpack failed: {exc}") from exc
292
293     # -----
294     # Check 3: magic identifier
295     # -----
296     if magic != MAGIC:
297         raise ParseError(
298             f"Bad magic bytes: got {magic!r}, expected {MAGIC!r}."
299         )
300
301     # -----
302     # Check 4: version
303     # -----
304     if version != VERSION:
305         raise ParseError(
306             f"Unsupported protocol version: {version:#04x} "
307             f"(this implementation supports {VERSION:#04x} only). "
308         )
309
310     # -----
311     # Check 5: reserved flags
312     # -----
313     if flags != 0x00:
314         raise ParseError(
315             f"Non-zero flags byte: {flags:#04x}. "
316             "Flags are reserved and must be 0x00 in protocol v1."
317         )
318
319     # -----
320     # Check 6: declared length vs. actual length
321     # -----
322     expected_total: int = HEADER_SIZE + ct_len + TAG_SIZE
323     if len(raw) != expected_total:
324         raise ParseError(
325             f"Length mismatch: header declares {ct_len}-byte ciphertext + "
326             f"{TAG_SIZE}-byte tag = {expected_total} bytes total, "
327             f"but envelope is {len(raw)} bytes."
328         )
329
330     # -----
331     # Slice ciphertext and tag (safe – lengths validated above)
332     # -----
333     ct_start: int = HEADER_SIZE
334     ct_end: int = ct_start + ct_len
335     tag_end: int = ct_end + TAG_SIZE

```

```

336
337     ciphertext: bytes = raw[ct_start:ct_end]
338     tag: bytes = raw[ct_end:tag_end]
339
340     return sequence, nonce, ciphertext, tag
341
342
343 # -----
344 # AEAD wrappers
345 # -----
346
347
348 def encrypt(key: bytes, plaintext: bytes, nonce: bytes) -> tuple[bytes, bytes]:
349     """Encrypt plaintext with AES-256-GCM.
350
351     Delegates entirely to
352     ``cryptography.hazmat.primitives.ciphers.aead.AESGCM``.
353     No AES or GCM arithmetic is performed in this module.
354
355     ``AESGCM.encrypt()`` returns ``ciphertext || tag`` concatenated.
356     This function splits the combined output into separate ``(ciphertext, tag)``
357     fields to match the envelope wire format (TAD Section 4.1).
358
359     Parameters
360     -----
361     key : bytes
362         32-byte AES-256 symmetric key (current ratchet key Ki, obtained
363         from :func:`dke_engine.outbound_params`). Must be exactly
364         :data:`KEY_SIZE` bytes.
365     plaintext : bytes
366         The message bytes to encrypt. May be empty.
367     nonce : bytes
368         12-byte unique nonce for this message. MUST be generated by the
369         caller using ``os.urandom(12)`` immediately before each call.
370         Reusing a nonce under the same key is catastrophic for AES-GCM
371         security. Must be exactly :data:`NONCE_SIZE` bytes.
372
373     Returns
374     -----
375     tuple[bytes, bytes]
376         ``(ciphertext, tag)`` where:
377
378         * ``ciphertext`` has the same length as ``plaintext``.
379         * ``tag`` is exactly :data:`TAG_SIZE` (16) bytes.
380
381     Raises
382     -----
383     ValueError
384         If ``key`` is not 32 bytes or ``nonce`` is not 12 bytes.
385
386     Examples
387     -----
388     >>> import os
389     >>> key = os.urandom(32)
390     >>> nonce = os.urandom(12)
391     >>> ct, tag = encrypt(key, b"hello world", nonce)
392     >>> assert len(tag) == 16
393     >>> assert len(ct) == len(b"hello world")
394
395     TAD Reference: Section 3.3 – encrypt()
396                     Section 5.2 – Confidentiality, Integrity & Authenticity
397                     Section 5.4 – AES-256-GCM algorithm choice
398     """
399     if len(key) != KEY_SIZE:
400         raise ValueError(
401             f"key must be {KEY_SIZE} bytes, got {len(key)}."
402         )

```

```

403     if len(nonce) != NONCE_SIZE:
404         raise ValueError(
405             f"nonce must be {NONCE_SIZE} bytes, got {len(nonce)}."
406         )
407
408     aesgcm = AESGCM(key)
409     # AESGCM.encrypt(nonce, data, aad) returns ciphertext || tag (tag is last 16 B)
410     combined: bytes = aesgcm.encrypt(nonce, plaintext, None)
411
412     ciphertext: bytes = combined[:-TAG_SIZE]
413     tag: bytes = combined[-TAG_SIZE:]
414     return ciphertext, tag
415
416
417 def decrypt(
418     key: bytes,
419     ciphertext: bytes,
420     nonce: bytes,
421     tag: bytes,
422 ) -> bytes:
423     """Decrypt and authenticate a ciphertext with AES-256-GCM.
424
425     Delegates entirely to
426     ``cryptography.hazmat.primitives.ciphers.aead.AESGCM``.
427     No AES or GCM arithmetic is performed in this module.
428
429     Concatenates ``ciphertext + tag`` before passing to
430     ``AESGCM.decrypt()`` which requires the combined form. If the tag
431     does not authenticate the ciphertext, ``AESGCM.decrypt()`` raises
432     ``cryptography.exceptions.InvalidTag`` **before returning any bytes**.
433     This function propagates that exception unchanged – it NEVER returns
434     plaintext on an authentication failure (fail-closed, TAD §7.1).
435
436     Parameters
437     -----
438     key : bytes
439         32-byte AES-256 symmetric key (current ratchet key Ki, obtained
440         from :func:`dke_engine.inbound_key`). Must be exactly
441         :data:`KEY_SIZE` bytes.
442     ciphertext : bytes
443         The ciphertext field from :func:`decode_envelope`.
444     nonce : bytes
445         The 12-byte nonce field from :func:`decode_envelope`.
446     tag : bytes
447         The 16-byte authentication tag field from :func:`decode_envelope`.
448         Must be exactly :data:`TAG_SIZE` bytes.
449
450     Returns
451     -----
452     bytes
453         Verified plaintext. Returned only when authentication succeeds.
454
455     Raises
456     -----
457     cryptography.exceptions.InvalidTag
458         If the authentication tag does not match the ciphertext.
459         No plaintext (or partial plaintext) is ever returned in this case.
460     ValueError
461         If ``key`` is not 32 bytes, ``nonce`` is not 12 bytes, or
462         ``tag`` is not 16 bytes.
463
464     Examples
465     -----
466     >>> import os
467     >>> key = os.urandom(32)
468     >>> nonce = os.urandom(12)
469     >>> ct, tag = encrypt(key, b"hello world", nonce)

```

```
470     >>> plaintext = decrypt(key, ct, nonce, tag)
471     >>> assert plaintext == b"hello world"
472
473     TAD Reference: Section 3.3 – decrypt()
474                     Section 7 – Failure Modes (Authentication tag mismatch)
475                     Section 7.1 – Fail-closed policy
476     """
477     if len(key) != KEY_SIZE:
478         raise ValueError(
479             f"key must be {KEY_SIZE} bytes, got {len(key)}."
480         )
481     if len(nonce) != NONCE_SIZE:
482         raise ValueError(
483             f"nonce must be {NONCE_SIZE} bytes, got {len(nonce)}."
484         )
485     if len(tag) != TAG_SIZE:
486         raise ValueError(
487             f"tag must be {TAG_SIZE} bytes, got {len(tag)}."
488         )
489
490     aesgcm = AESGCM(key)
491     # AESGCM.decrypt expects ciphertext || tag concatenated
492     combined: bytes = ciphertext + tag
493     # Raises cryptography.exceptions.InvalidTag on authentication failure;
494     # no plaintext is ever returned in that case.
495     plaintext: bytes = aesgcm.decrypt(nonce, combined, None)
496     return plaintext
```

```

1  #!/usr/bin/env python3
2  """
3  main_chat.py – CLI Orchestration Entry Point
4  =====
5  TAD Reference: Section 3.4, Section 6
6
7  Responsibility
8  -----
9  Top-level CLI entry point. Wires together the full session lifecycle:
10
11  1. Interactive REPL with commands: start, send, status, end, help, quit.
12  2. TCP socket transport (listener or connector).
13  3. Ed25519 identity key generation and fingerprint display on startup.
14  4. Authenticated X25519 public-key exchange during handshake:
15     each party sends  ecdh_pub (32 B) || ed25519_sig (64 B) = 96 B.
16  5. Signature verification before ECDH – rejects MITM attempts.
17  6. Shared secret → HKDF → K0 → DKESessionState seeding.
18  7. Steady-state bidirectional encrypted message loop (TAD Section 6.2).
19
20  Usage
21  -----
22  Run two terminals side-by-side:
23
24  Terminal A (listener / Alice):
25      python main_chat.py
26      >> start --listen 9000
27
28  Terminal B (connector / Bob):
29      python main_chat.py
30      >> start --connect 127.0.0.1:9000
31
32  To enable MITM protection, pass the peer's identity key (printed at
33  their startup) as a hex string:
34
35      >> start --listen 9000 --peer-key <alice_identity_hex>
36      >> start --connect 127.0.0.1:9000 --peer-key <bob_identity_hex>
37
38  Once connected, both sides can type:
39      >> send Hello!
40      >> status
41      >> end
42
43  Session Establishment Sequence (TAD Section 6.1 – with Ed25519 auth)
44  -----
45  Alice (listener)                                Bob (connector)
46  generate_identity_keypair()                      generate_identity_keypair()
47  generate_keypair()                               generate_keypair()
48  sig_a = sign(id_priv_a, a_pub)                   sig_b = sign(id_priv_b, b_pub)
49  send(a_pub || sig_a) 96 B ██████████          recv 96 B
50                                                    verify(id_pub_a, a_pub, sig_a) ← aborts if invalid
51  recv 96 B ██████████                          send(b_pub || sig_b) 96 B
52  verify(id_pub_b, b_pub, sig_b)                   shared = ECDH(b_priv, a_pub)
53  shared = ECDH(a_priv, b_pub)                      K0 = HKDF(shared, SESSION_INFO)
54  K0 = HKDF(shared, SESSION_INFO)                   [Both sides hold identical K0]
55  [Both sides hold identical K0]
56
57  Transport Framing (envelope receive)
58  -----
59  Envelopes are self-describing:
60  1. Read exactly HEADER_SIZE (30) bytes.
61  2. Parse ct_len from header bytes [26:30] (uint32 big-endian).
62  3. Read exactly ct_len + TAG_SIZE (16) more bytes.
63  4. Concatenate and pass to protocol.decode_envelope().
64
65  No extra length-prefix framing is required because the header already
66  encodes the ciphertext length.
67

```

```

68 Error Handling Philosophy (TAD Section 7.1)
69 -----
70 * Signature verification failure → session rejected immediately with
71 "[!] Identity verification failed – possible MITM attempt".
72 * Decryption / tag failures → message dropped, metadata logged, session
73 continues.
74 * Sequence mismatch → message dropped, session continues.
75 * Peer disconnect → session state discarded; new 'start' required.
76 """
77
78 from __future__ import annotations
79
80 import os
81 import socket
82 import struct
83 import sys
84 import textwrap
85 import threading
86 from typing import Optional
87
88 try:
89     sys.stdout.reconfigure(line_buffering=True)
90 except Exception:
91     pass
92
93 from cryptography.exceptions import InvalidTag, InvalidSignature
94
95 # -----
96 # Project modules
97 # -----
98 import crypto_core
99 import dke_engine
100 import protocol
101 from crypto_core import (
102     SESSION_INFO,
103     PUBLIC_KEY_SIZE,
104     IDENTITY_PUBLIC_KEY_SIZE,
105     SIGNATURE_SIZE,
106     generate_identity_keypair,
107     sign_public_key,
108     verify_public_key,
109 )
110 from dke_engine import (
111     DKESessionState,
112     SequenceError,
113     inbound_key,
114     outbound_params,
115     rotate_receive,
116     rotate_send,
117 )
118 from protocol import (
119     HEADER_SIZE,
120     TAG_SIZE,
121     ParseError,
122     decode_envelope,
123     encode_envelope,
124     encrypt,
125     decrypt,
126 )
127
128 # -----
129 # Version / Banner
130 # -----
131
132 _VERSION = "1.0"
133 _BANNER = (
134     "DKE Messaging Engine v" + _VERSION + "\n"

```

```

135     "End-to-end encrypted chat with Dynamic Key Evolution.\n"
136     "Ed25519 identity authentication enabled (MITM protection).\n"
137     "Type 'help' for available commands."
138 )
139
140 _PROMPT = "> "
141
142 # =====
143 # Centralized User-Facing Strings (Frontend Specification Sections 5, 6, 7)
144 # =====
145 MSG_WAITING_LISTEN = "[*] Waiting for peer to connect on port {port} ..."
146 MSG_CONNECTING = "[*] Connecting to {host}:{port} ..."
147 MSG_PEER_CONNECTED = "[*] Peer connected. Exchanging public keys ..."
148 MSG_SESSION_ESTABLISHED = "[+] Secure session established. Type your message and press Enter."
149 MSG_SESSION_ENDED = "[*] Session ended. Key material discarded."
150 MSG_IDENTITY_KEY = "[*] Your identity key (share this with your peer out-of-band):\n    {hex}"
151 MSG_PEER_KEY_WARN = (
152     "[!] No --peer-key supplied – identity verification DISABLED for this session.\n"
153     "    MITM protection is NOT active. Provide --peer-key <hex> to enable it."
154 )
155 MSG_PEER_KEY_OK = "[*] Peer identity key accepted. MITM protection ACTIVE."
156
157 ERR_AUTH_FAILED = "[!] A message failed integrity verification and was discarded."
158 ERR_OUT_OF_ORDER = "[!] Unexpected message order detected; message discarded. Session remains active."
159 ERR_PEER_LOST = "[!] Peer connection lost. Session ended; start a new session to continue."
160 ERR_UNKNOWN_CMD = "[!] Unknown command. Type 'help' to see available commands."
161 ERR_NO_SESSION_SEND = "[!] No active session. Use 'start --listen' or 'start --connect' first."
162 ERR_ALREADY_ACTIVE = "[!] A session is already active. Use 'end' first."
163 ERR_NO_SESSION_END = "[!] No active session to end."
164 ERR_NO_SESSION_STATUS = "[!] No active session."
165 ERR_IDENTITY_VERIFY = "[!] Identity verification failed – possible MITM attempt"
166
167 # -----
168 # Identity key – generated once at process startup, never written to disk
169 # -----
170 # Each party generates a fresh Ed25519 identity keypair on every launch.
171 # The public key is printed as a hex fingerprint so the peer can verify
172 # it out-of-band (analogous to Signal's "safety numbers").
173 _identity_private, _identity_public = generate_identity_keypair()
174
175 # -----
176 # Global session state (protected by _lock)
177 # -----
178
179 _lock: threading.Lock = threading.Lock()
180 _session_state: Optional[DKESessionState] = None
181 _session_sock: Optional[socket.socket] = None
182 _session_role: Optional[str] = None          # "alice" or "bob" – display only
183 _session_peer: str = ""                     # "host:port" for display
184 _session_active: threading.Event = threading.Event()
185 _recv_thread: Optional[threading.Thread] = None
186
187
188 # =====
189 # Low-level transport helpers
190 # =====
191
192
193 def _recv_exactly(sock: socket.socket, n: int) -> bytes:
194     """Read exactly `n` bytes from `sock`, blocking until available.
195
196     Parameters
197     -----
198     sock : socket.socket
199         An open, connected TCP socket.
200     n : int
201         Number of bytes to read.

```

```

202
203     Returns
204     -----
205     bytes
206         Exactly ``n`` bytes.
207
208     Raises
209     -----
210     ConnectionError
211         If the connection closes before ``n`` bytes are received.
212
213     TAD Reference: Section 2.2 – Transport (dumb pipe)
214     """
215     buf = bytearray()
216     while len(buf) < n:
217         chunk = sock.recv(n - len(buf))
218         if not chunk:
219             raise ConnectionError("Peer closed the connection.")
220         buf.extend(chunk)
221     return bytes(buf)
222
223
224 MAX_CIPHertext_SIZE: int = 64 * 1024 * 1024 # 64 MB maximum ciphertext payload safety cap
225
226
227 def _recv_envelope(sock: socket.socket) -> bytes:
228     """Read exactly one complete DKE envelope from the socket.
229
230     Uses the envelope's own header to determine the total byte count –
231     no external length-prefix framing needed.
232
233     Steps:
234     1. Read HEADER_SIZE (30) bytes.
235     2. Parse ``ct_len`` from bytes [26:30] (uint32 big-endian).
236     3. Read ``ct_len + TAG_SIZE`` more bytes.
237
238     Returns
239     -----
240     bytes
241         Complete raw envelope ready for ``protocol.decode_envelope()``.
242
243     Raises
244     -----
245     ConnectionError
246         If the connection closes mid-read.
247     ParseError
248         If ``ct_len`` exceeds the maximum safe payload limit.
249
250     TAD Reference: Section 4.1 – Envelope Structure
251     """
252     header_bytes = _recv_exactly(sock, HEADER_SIZE)
253     (ct_len,) = struct.unpack_from("!I", header_bytes, offset=26)
254     if ct_len > MAX_CIPHertext_SIZE:
255         raise ParseError(
256             f"Declared ciphertext length {ct_len} exceeds maximum safety limit "
257             f"({MAX_CIPHertext_SIZE} bytes).")
258     )
259     rest = _recv_exactly(sock, ct_len + TAG_SIZE)
260     return header_bytes + rest
261
262
263 def _send_envelope(sock: socket.socket, envelope: bytes) -> None:
264     """Send a complete envelope over the socket.
265
266     Uses ``sendall()`` to guarantee the full byte sequence is written.
267
268     Raises

```

```

269     -----
270     OSError
271         If the send fails (connection broken).
272
273     TAD Reference: Section 2.2 – Transport (dumb pipe)
274     """
275     sock.sendall(envelope)
276
277
278 # =====
279 # Handshake
280 # =====
281
282
283 def _handshake_listener(
284     port: int,
285     peer_identity_pub: bytes | None,
286 ) -> tuple[socket.socket, DKESessionState, str]:
287     """Listen for one incoming connection and perform the authenticated DKE handshake.
288
289     Role: Alice (listener).
290
291     Handshake order (Ed25519-authenticated):
292     1. Bind and listen on ``port``.
293     2. Accept the first incoming connection (one-shot server).
294     3. Generate X25519 ephemeral keypair.
295     4. Sign local ECDH public key with Ed25519 identity private key.
296     5. Send ``ecdh_pub`` (32 B) || signature (64 B) = 96 bytes to peer.
297     6. Receive peer's 96-byte payload; split into peer_ecdh_pub + peer_sig.
298     7. Verify peer_sig against peer_identity_pub (if supplied); abort on failure.
299     8. ECDH → HKDF → K0 → DKESessionState.
300
301     Parameters
302     -----
303     port : int
304         TCP port to listen on (0-65535).
305     peer_identity_pub : bytes or None
306         The peer's 32-byte Ed25519 identity public key (obtained out-of-band).
307         ``None`` disables signature verification for this session (demo mode).
308
309     Returns
310     -----
311     tuple[socket.socket, DKESessionState, str]
312         ``(conn, session_state, peer_addr_string)``
313
314     Raises
315     -----
316     OSError
317         If the socket cannot be bound.
318     ConnectionError
319         If signature verification fails or ECDH key exchange fails.
320
321     TAD Reference: Section 6.1 – Session Establishment
322                   Section 4.3 – Public Key Exchange Format
323     """
324     try:
325         server_sock = socket.create_server(("", port), family=socket.AF_INET6, dualstack_ipv6=True)
326     except Exception:
327         server_sock = socket.create_server(("", port))
328
329     print(MSG_WAITING_LISTEN.format(port=port), flush=True)
330     conn, addr = server_sock.accept()
331     server_sock.close() # one-shot: reject further connections
332
333     peer_str = f"{addr[0]}:{addr[1]}"
334     print(MSG_PEER_CONNECTED, flush=True)
335

```



```

470
471     1. Read one complete envelope from the socket.
472     2. Decode (parse + validate wire format).
473     3. Validate sequence number (reject replays / reorders).
474     4. Decrypt and authenticate (AES-256-GCM).
475     5. Display plaintext.
476     6. Rotate key + increment inbound seq counter.
477
478     All errors are logged with metadata only – no plaintext is ever
479     written to the log on failure (TAD Section 7.1).
480
481     TAD Reference: Section 6.2 – Steady-State Receive
482                   Section 6.3 – Decryption Failure Path
483                   Section 7 – Failure Modes & Error Handling
484     """
485     global _session_state, _session_sock, _session_role
486
487     while _session_active.is_set():
488         try:
489             raw = _recv_envelope(_session_sock)
490         except ConnectionError:
491             if _session_active.is_set():
492                 print(
493                     f"\n{ERR_PEER_LOST}\n{_PROMPT}",
494                     end="",
495                     flush=True,
496                 )
497                 _session_active.clear()
498             return
499         except OSError:
500             return
501         except ParseError:
502             print(
503                 f"\n{ERR_AUTH_FAILED}\n{_PROMPT}",
504                 end="",
505                 flush=True,
506             )
507             continue
508
509         try:
510             seq, nonce, ct, tag = decode_envelope(raw)
511         except ParseError:
512             # Structural error – drop message, log metadata only, continue
513             print(
514                 f"\n{ERR_AUTH_FAILED}\n{_PROMPT}",
515                 end="",
516                 flush=True,
517             )
518             continue
519
520         try:
521             with _lock:
522                 if _session_state is None or not _session_active.is_set():
523                     return
524                 key = inbound_key(_session_state, seq)
525                 plaintext = decrypt(key, ct, nonce, tag) # raises on failure
526                 rotate_receive(_session_state, nonce, seq)
527         except SequenceError:
528             print(
529                 f"\n{ERR_OUT_OF_ORDER}\n{_PROMPT}",
530                 end="",
531                 flush=True,
532             )
533             continue
534         except InvalidTag:
535             # Authentication failed – fail closed, never display partial data
536             print(

```



```

671
672
673 def _cmd_send(parts: list[str]) -> None:
674     """Encrypt and send a plaintext message to the peer.
675
676     Steps (TAD Section 6.2 – Steady-State Send):
677         1. Snapshot current key and outbound seq from session state.
678         2. Generate fresh 12-byte nonce via os.urandom(12).
679         3. Encrypt plaintext with AES-256-GCM.
680         4. Encode into a DKE binary envelope.
681         5. Send over TCP socket.
682         6. Rotate key and increment outbound seq counter.
683
684     TAD Reference: Section 3.4 – Steady-State Loop
685                   Section 6.2 – Steady-State Send
686     """
687     if not _session_active.is_set():
688         print(ERR_NO_SESSION_SEND)
689         return
690     if not parts:
691         print("[!] Usage: send <message>")
692         return
693
694     plaintext_str = " ".join(parts)
695     plaintext = plaintext_str.encode("utf-8")
696     nonce = os.urandom(12)
697
698     try:
699         with _lock:
700             key, seq = outbound_params(_session_state)
701             ct, tag = encrypt(key, plaintext, nonce)
702             envelope = encode_envelope(seq, nonce, ct, tag)
703             _send_envelope(_session_sock, envelope)
704             rotate_send(_session_state, nonce)
705     except OSError as exc:
706         print(f"[!] Send failed: {exc}")
707         _session_active.clear()
708         return
709
710     print(f"[you, seq {seq}] {plaintext_str}")
711
712
713 def _cmd_status() -> None:
714     """Display current session state and message counters (no key material).
715
716     Frontend Spec Reference: Section 4, Section 5.4, Section 7.
717     """
718     if not _session_active.is_set():
719         print(ERR_NO_SESSION_STATUS)
720         return
721
722     with _lock:
723         state = _session_state
724         sent_count = state.outbound.sequence_number
725         rcv_count = state.inbound.sequence_number
726         out_seq = sent_count + 1
727         in_seq = rcv_count + 1
728
729     print(
730         f"Session: ACTIVE\n"
731         f"Messages sent: {sent_count}   Messages received: {rcv_count}\n"
732         f"Current sequence (out): {out_seq}   Current sequence (in): {in_seq}"
733     )
734
735
736 def _cmd_end() -> None:
737     """Gracefully close the session and discard all session state.
```

```

738
739     Session state (keys, counters) is discarded entirely.
740     A new session requires a fresh key exchange.
741
742     TAD Reference: Section 7 – Failure Modes (Peer disconnects)
743     """
744     global _session_state, _session_sock, _session_role, _session_peer
745
746     if not _session_active.is_set():
747         print(ERR_NO_SESSION_END)
748         return
749
750     _session_active.clear()
751
752     with _lock:
753         if _session_sock:
754             try:
755                 _session_sock.shutdown(socket.SHUT_RDWR)
756                 _session_sock.close()
757             except OSError:
758                 pass
759             if _session_state is not None:
760                 _session_state.wipe() # Best-effort zeroization of remaining key material
761             _session_state = None
762             _session_sock = None
763             _session_role = None
764             _session_peer = ""
765
766     print(MSG_SESSION_ENDED)
767
768
769 def _cmd_help() -> None:
770     """Print the list of available commands."""
771     print(textwrap.dedent("""
772         Commands
773         -----
774         start --listen <port> [--peer-key <hex>]      Starts as session initiator, waiting for peer
775         start --connect <host:port> [--peer-key <hex>] Starts as joining party, connecting to peer
776         send <message>                                Encrypts and sends a message in an active session
777         status                                         Shows current session state and message counters (no key material)
778         end                                           Ends the session and discards all key material
779         help                                           Lists available commands
780         quit                                           Exit the program
781
782         MITM Protection
783         -----
784         Pass --peer-key <hex> with the peer's identity key (printed at their startup)
785         to enable Ed25519 signature verification. Without --peer-key, verification
786         is skipped and a warning is displayed.
787
788         Message display
789         -----
790         [you, seq N] <text>                          Message sent by you
791         [peer, seq N] <text>                          Message received from peer
792         [!] <text>                                     Error or warning
793         [*] <text>                                     Status / info
794         [+] <text>                                     Secure session established
795     """).strip())
796
797
798 # =====
799 # Main REPL
800 # =====
801
802
803 def main() -> None:
804     """Interactive REPL entry point.

```

```

805
806 Prints the banner, displays the local Ed25519 identity key fingerprint,
807 then loops reading user input. If command-line arguments are provided
808 (e.g. 'start --listen 5050' or 'start --connect localhost:5050'),
809 executes the command initially before continuing into the interactive REPL.
810
811 TAD Reference: Section 3.4 – main_chat.py overview
812 """
813 print(_BANNER)
814 print()
815 # Print identity fingerprint so the peer can verify it out-of-band
816 print(MSG_IDENTITY_KEY.format(hex=_identity_public.hex()), flush=True)
817 print()
818 # Parse initial command-line arguments if provided
819 # e.g.: python main_chat.py start --listen 5050
820 #       python main_chat.py start --connect localhost:5050
821 #       python main_chat.py --listen 5050
822 if len(sys.argv) > 1:
823     first = sys.argv[1].lower()
824     if first == "start":
825         _cmd_start(sys.argv[2:])
826     elif first in ("--listen", "-l", "--connect", "-c"):
827         _cmd_start(sys.argv[1:])
828     elif first in ("help", "--help", "-h"):
829         _cmd_help()
830         sys.exit(0)
831     elif first == "status":
832         _cmd_status()
833     elif first in ("quit", "exit", "q"):
834         sys.exit(0)
835     else:
836         print(ERR_UNKNOWN_CMD)
837
838 while True:
839     try:
840         line = input(_PROMPT).strip()
841     except (EOFError, KeyboardInterrupt):
842         print()
843         _cmd_end()
844         print("[*] Goodbye.")
845         sys.exit(0)
846
847     if not line:
848         continue
849
850     tokens = line.split()
851     cmd = tokens[0].lower()
852     args = tokens[1:]
853
854     if cmd == "start":
855         _cmd_start(args)
856     elif cmd == "send":
857         _cmd_send(args)
858     elif cmd == "status":
859         _cmd_status()
860     elif cmd == "end":
861         _cmd_end()
862     elif cmd in ("help", "?"):
863         _cmd_help()
864     elif cmd in ("quit", "exit", "q"):
865         _cmd_end()
866         print(" Goodbye.")
867         sys.exit(0)
868     else:
869         print(ERR_UNKNOWN_CMD)
870
871

```

```
872 if __name__ == "__main__":  
873     main()
```

```

1 """
2 tests/test_dke.py – Automated Test Suite
3 =====
4 TAD Reference: Section 3.5
5
6 Responsibility
7 -----
8 Correctness and security regression tests for the DKE Messaging Engine.
9 Run with::
10
11     pytest tests/
12
13 All test functions follow the naming convention ``test_<behaviour>``
14 and use only public module APIs.
15
16 Design note on two-chain state
17 -----
18 ``DKESessionState`` now holds two independent ``DirectionState`` objects:
19
20     state.outbound – key + seq for messages *sent* by this party
21     state.inbound  – key + seq for messages *received* by this party
22
23 The cross-matching invariant after any exchange:
24     sender.outbound.current_key == receiver.inbound.current_key
25
26 Test Coverage Matrix (TAD Section 3.5)
27 -----
28 +-----+-----+
29 | Test case                                | TAD property verified |
30 +-----+-----+
31 | ECDH symmetry                           | Shared secret derivation |
32 | HKDF output length                       | Key derivation contract |
33 | HKDF domain separation via info string   | Key derivation contract |
34 | advance_key changes the key              | Forward secrecy         |
35 | advance_key is deterministic             | Ratchet correctness     |
36 | advance_key is non-reversible            | Key Non-reversibility   |
37 | Ratchet chain: all steps unique          | Forward secrecy         |
38 | validate_sequence rejects out-of-order   | Replay prevention       |
39 | validate_sequence rejects replay (seq too low) | Replay prevention       |
40 | validate_sequence accepts correct seq    | Sequence counter        |
41 | DKESessionState initial counter values   | State correctness       |
42 | rotate_send advances outbound key & erases old | Memory safety           |
43 | rotate_receive validates + advances inbound key | State correctness       |
44 | rotate_receive rejects bad seq (state unchanged) | Fail-closed ratchet     |
45 | Outbound chain independent of inbound chain | Directional isolation   |
46 | Cross-match: sender.outbound == receiver.inbound | End-to-end DKE         |
47 | Full bidirectional session key agreement | End-to-end DKE         |
48 | encrypt/decrypt round-trip               | AEAD confidentiality    |
49 | encrypt: empty plaintext                 | Edge case               |
50 | tampered ciphertext -> InvalidTag         | Integrity (GCM tag)     |
51 | tampered tag -> InvalidTag               | Authenticity (GCM tag)  |
52 | wrong key -> InvalidTag                  | Key binding             |
53 | decrypt never returns on tag failure      | Fail-closed policy      |
54 | encode_envelope / decode_envelope round-trip | Wire format            |
55 | decode rejects input < 46 bytes           | Malformed envelope     |
56 | decode rejects wrong magic               | Malformed envelope     |
57 | decode rejects unsupported version        | Malformed envelope     |
58 | decode rejects non-zero flags            | Malformed envelope     |
59 | decode rejects truncated ciphertext       | Malformed envelope     |
60 | decode rejects padded envelope           | Malformed envelope     |
61 | decode rejects mismatched ct_len field   | Malformed envelope     |
62 | Full pipeline: ECDH -> HKDF -> DKE -> AES-GCM | Integration             |
63 | Tampered envelope: decode OK, decrypt fails | Fail-closed policy      |
64 +-----+-----+
65 """
66
67 from __future__ import annotations

```

```

68
69 import os
70 import struct
71
72 import pytest
73 from cryptography.exceptions import InvalidTag, InvalidSignature
74
75 import sys
76 import pathlib
77 sys.path.insert(0, str(pathlib.Path(__file__).parent.parent))
78
79 from crypto_core import (
80     generate_keypair,
81     derive_shared_secret,
82     hkdf_derive,
83     SESSION_INFO,
84     PUBLIC_KEY_SIZE,
85     DEFAULT_KEY_LENGTH,
86     generate_identity_keypair,
87     sign_public_key,
88     verify_public_key,
89     IDENTITY_PUBLIC_KEY_SIZE,
90     SIGNATURE_SIZE,
91 )
92 from dke_engine import (
93     DirectionState,
94     DKESessionState,
95     SequenceError,
96     advance_key,
97     validate_sequence,
98     outbound_params,
99     inbound_key,
100     rotate_send,
101     rotate_receive,
102     KEY_SIZE,
103     NONCE_SIZE,
104 )
105 from protocol import (
106     encode_envelope,
107     decode_envelope,
108     encrypt,
109     decrypt,
110     ParseError,
111     MAGIC,
112     TAG_SIZE,
113     MIN_ENVELOPE_SIZE,
114     HEADER_SIZE,
115 )
116
117
118 # =====
119 # Shared fixtures
120 # =====
121
122
123 @pytest.fixture()
124 def keypair_alice():
125     """Fresh X25519 key pair for Alice."""
126     return generate_keypair()
127
128
129 @pytest.fixture()
130 def keypair_bob():
131     """Fresh X25519 key pair for Bob."""
132     return generate_keypair()
133
134

```

```

135 @pytest.fixture()
136 def shared_secret(keypair_alice, keypair_bob):
137     """Raw ECDH shared secret (same from both sides)."""
138     priv_a, pub_a = keypair_alice
139     priv_b, pub_b = keypair_bob
140     return derive_shared_secret(priv_a, pub_b)
141
142
143 @pytest.fixture()
144 def k0(shared_secret):
145     """Initial 32-byte AES-256 key derived from the shared secret."""
146     return hkdf_derive(shared_secret, info=SESSION_INFO)
147
148
149 @pytest.fixture()
150 def session_states(k0):
151     """Pair of DKESessionState objects both seeded with K0 (Alice and Bob)."""
152     return DKESessionState.from_k0(k0), DKESessionState.from_k0(k0)
153
154
155 @pytest.fixture()
156 def aes_key():
157     """Random 32-byte AES-256 key."""
158     return os.urandom(KEY_SIZE)
159
160
161 @pytest.fixture()
162 def nonce():
163     """Random 12-byte AES-GCM nonce."""
164     return os.urandom(NONCE_SIZE)
165
166
167 @pytest.fixture()
168 def encrypted_hello(aes_key, nonce):
169     """Pre-encrypted b'hello world' and its tag."""
170     ct, tag = encrypt(aes_key, b"hello world", nonce)
171     return ct, tag
172
173
174 # =====
175 # crypto_core tests
176 # =====
177
178
179 class TestCryptoCore:
180
181     def test_ecdh_shared_secret_symmetry(self, keypair_alice, keypair_bob):
182         """Both sides of X25519 ECDH must derive an identical shared secret.
183
184         Verifies::
185
186             derive_shared_secret(a_priv, b_pub) == derive_shared_secret(b_priv, a_pub)
187
188         TAD Reference: Section 3.5 – ECDH symmetry
189                     Section 5.1 – Key Hierarchy
190
191         """
192         priv_a, pub_a = keypair_alice
193         priv_b, pub_b = keypair_bob
194
195         secret_a = derive_shared_secret(priv_a, pub_b)
196         secret_b = derive_shared_secret(priv_b, pub_a)
197
198         assert secret_a == secret_b, (
199             "ECDH must be symmetric: A's view of the secret must equal B's view."
200         )
201         assert len(secret_a) == PUBLIC_KEY_SIZE

```

```

202 def test_ecdh_different_peers_yield_different_secrets(self, keypair_alice):
203     """Different peers must produce different shared secrets."""
204     priv_a, _ = keypair_alice
205     _, pub_b1 = generate_keypair()
206     _, pub_b2 = generate_keypair()
207
208     secret1 = derive_shared_secret(priv_a, pub_b1)
209     secret2 = derive_shared_secret(priv_a, pub_b2)
210
211     assert secret1 != secret2
212
213 def test_ecdh_rejects_short_peer_key(self, keypair_alice):
214     """derive_shared_secret() must reject peer keys that are not 32 bytes."""
215     priv_a, _ = keypair_alice
216     with pytest.raises(ValueError, match="32 bytes"):
217         derive_shared_secret(priv_a, b"tooshort")
218
219 def test_hkdf_derive_produces_correct_length(self, shared_secret):
220     """hkdf_derive() must return exactly ``length`` bytes (default 32).
221
222     TAD Reference: Section 3.1 – hkdf_derive()
223     """
224     key = hkdf_derive(shared_secret, info=SESSION_INFO)
225     assert len(key) == DEFAULT_KEY_LENGTH
226
227 def test_hkdf_custom_length(self, shared_secret):
228     """hkdf_derive() must honour a non-default length argument."""
229     key = hkdf_derive(shared_secret, info=SESSION_INFO, length=64)
230     assert len(key) == 64
231
232 def test_hkdf_different_info_produces_different_keys(self, shared_secret):
233     """Different ``info`` strings MUST yield different derived keys (domain separation).
234
235     TAD Reference: Section 3.1 – Design Constraints (info parameter)
236     """
237     key_a = hkdf_derive(shared_secret, info="context-a")
238     key_b = hkdf_derive(shared_secret, info="context-b")
239
240     assert key_a != key_b
241
242 def test_hkdf_same_inputs_same_output(self, shared_secret):
243     """hkdf_derive() is deterministic: same inputs → same key."""
244     k1 = hkdf_derive(shared_secret, info=SESSION_INFO)
245     k2 = hkdf_derive(shared_secret, info=SESSION_INFO)
246     assert k1 == k2
247
248 def test_hkdf_rejects_empty_secret(self):
249     """hkdf_derive() must raise ValueError for an empty shared secret."""
250     with pytest.raises(ValueError):
251         hkdf_derive(b"", info=SESSION_INFO)
252
253 def test_both_sides_derive_same_k0(self, keypair_alice, keypair_bob):
254     """Full handshake: both sides must arrive at the same K0.
255
256     TAD Reference: Section 5.1 – Key Hierarchy
257                   Section 6.1 – Session Establishment
258     """
259     priv_a, pub_a = keypair_alice
260     priv_b, pub_b = keypair_bob
261
262     k0_a = hkdf_derive(derive_shared_secret(priv_a, pub_b), info=SESSION_INFO)
263     k0_b = hkdf_derive(derive_shared_secret(priv_b, pub_a), info=SESSION_INFO)
264
265     assert k0_a == k0_b, "Both parties must independently derive the same K0."
266
267
268 # =====

```

```

269 # dke_engine tests
270 # =====
271
272
273 class TestAdvanceKey:
274
275     def test_advance_key_changes_key(self):
276         """advance_key() must return a key that differs from the input key.
277
278         TAD Reference: Section 3.5 – Key rotation non-reversibility
279         """
280         k0 = os.urandom(KEY_SIZE)
281         nonce = os.urandom(NONCE_SIZE)
282         k1 = advance_key(k0, nonce, sequence=0)
283
284         assert k1 != k0
285         assert len(k1) == KEY_SIZE
286
287     def test_advance_key_deterministic(self):
288         """advance_key() with identical inputs must produce identical output.
289
290         Both sides independently compute the same next key.
291
292         TAD Reference: Section 3.2 – Key Evolution Formula
293         """
294         k0 = os.urandom(KEY_SIZE)
295         nonce = os.urandom(NONCE_SIZE)
296
297         k1_alice = advance_key(k0, nonce, sequence=0)
298         k1_bob = advance_key(k0, nonce, sequence=0)
299
300         assert k1_alice == k1_bob
301
302     def test_advance_key_is_non_reversible(self):
303         """Given Ki+1, it must be impossible to recover Ki.
304
305         TAD Reference: Section 5.2 – Key Non-reversibility
306         """
307         k0 = os.urandom(KEY_SIZE)
308         nonce = os.urandom(NONCE_SIZE)
309         k1 = advance_key(k0, nonce, sequence=0)
310
311         # Attempting forward advancement from k1 must not reproduce k0
312         k2 = advance_key(k1, nonce, sequence=0)
313         assert k2 != k0
314
315         k2_alt = advance_key(k1, nonce, sequence=1)
316         assert k2_alt != k0
317
318     def test_advance_key_all_steps_unique(self):
319         """Every step of a 10-message ratchet chain must produce a unique key."""
320         key = os.urandom(KEY_SIZE)
321         seen: set[bytes] = {key}
322         nonce = os.urandom(NONCE_SIZE)
323
324         for seq in range(10):
325             key = advance_key(key, nonce, sequence=seq)
326             assert key not in seen, f"Ratchet step {seq} produced a duplicate key."
327             seen.add(key)
328
329     def test_advance_key_different_nonces_yield_different_keys(self):
330         """Same key + seq but different nonces must produce different next keys."""
331         k0 = os.urandom(KEY_SIZE)
332         nonce_a, nonce_b = os.urandom(NONCE_SIZE), os.urandom(NONCE_SIZE)
333
334         k1_a = advance_key(k0, nonce_a, sequence=0)
335         k1_b = advance_key(k0, nonce_b, sequence=0)

```

```

336         assert k1_a != k1_b
337
338     def test_advance_key_rejects_wrong_key_size(self):
339         """advance_key() must raise ValueError if key is not 32 bytes."""
340         with pytest.raises(ValueError, match="32 bytes"):
341             advance_key(b"tooshort", os.urandom(NONCE_SIZE), 0)
342
343     def test_advance_key_rejects_wrong_nonce_size(self):
344         """advance_key() must raise ValueError if nonce is not 12 bytes."""
345         with pytest.raises(ValueError, match="12 bytes"):
346             advance_key(os.urandom(KEY_SIZE), b"short", 0)
347
348     def test_advance_key_rejects_negative_sequence(self):
349         """advance_key() must raise ValueError for a negative sequence number."""
350         with pytest.raises(ValueError, match="non-negative"):
351             advance_key(os.urandom(KEY_SIZE), os.urandom(NONCE_SIZE), -1)
352
353
354     class TestValidateSequence:
355
356     def test_validate_sequence_accepts_correct(self):
357         """validate_sequence() must not raise when received == expected."""
358         validate_sequence(expected=0, received=0)
359         validate_sequence(expected=42, received=42)
360
361     def test_validate_sequence_rejects_out_of_order(self):
362         """validate_sequence() must raise SequenceError when received > expected.
363
364         TAD Reference: Section 7 – Failure Modes (Out-of-order)
365         """
366         with pytest.raises(SequenceError):
367             validate_sequence(expected=3, received=4)
368
369     def test_validate_sequence_rejects_replay(self):
370         """validate_sequence() must raise SequenceError when received < expected.
371
372         TAD Reference: Section 7 – Failure Modes (Replay)
373         """
374         with pytest.raises(SequenceError):
375             validate_sequence(expected=5, received=4)
376
377     def test_validate_sequence_rejects_zero_whenexpecting_nonzero(self):
378         """Receiving seq=0 after messages have been processed must be rejected."""
379         with pytest.raises(SequenceError):
380             validate_sequence(expected=10, received=0)
381
382
383     class TestDKESessionState:
384
385     def test_from_k0_creates_two_direction_states(self, k0):
386         """from_k0() must populate both outbound and inbound DirectionState objects."""
387         state = DKESessionState.from_k0(k0)
388         assert isinstance(state.outbound, DirectionState)
389         assert isinstance(state.inbound, DirectionState)
390
391     def test_from_k0_both_keys_match_input(self, k0):
392         """Both outbound and inbound keys must equal K0 after from_k0()."""
393         state = DKESessionState.from_k0(k0)
394         assert bytes(state.outbound.current_key) == k0
395         assert bytes(state.inbound.current_key) == k0
396
397     def test_from_k0_initial_sequence_counters(self, k0):
398         """Both outbound and inbound sequence numbers must start at 0."""
399         state = DKESessionState.from_k0(k0)
400         assert state.outbound.sequence_number == 0
401         assert state.inbound.sequence_number == 0
402

```

```

403 def test_from_k0_rejects_wrong_size(self):
404     """from_k0() must raise ValueError if k0 is not 32 bytes."""
405     with pytest.raises(ValueError, match="32 bytes"):
406         DKESessionState.from_k0(b"tooshort")
407
408 def test_rotate_send_advances_outbound_key(self, k0):
409     """rotate_send() must update outbound.current_key.
410
411     TAD Reference: Section 3.2 – Memory Safety Rule
412     """
413     state = DKESessionState.from_k0(k0)
414     key_before = bytes(state.outbound.current_key)
415     rotate_send(state, os.urandom(NONCE_SIZE))
416     assert bytes(state.outbound.current_key) != key_before
417
418 def test_rotate_send_does_not_touch_inbound_chain(self, k0):
419     """rotate_send() must leave inbound.current_key completely unchanged.
420
421     TAD Reference: Section 3.2 – Two Independent Ratchet Chains
422     """
423     state = DKESessionState.from_k0(k0)
424     inbound_key_before = bytes(state.inbound.current_key)
425     inbound_seq_before = state.inbound.sequence_number
426
427     rotate_send(state, os.urandom(NONCE_SIZE))
428
429     assert bytes(state.inbound.current_key) == inbound_key_before
430     assert state.inbound.sequence_number == inbound_seq_before
431
432 def test_rotate_send_erases_old_outbound_key(self, k0):
433     """rotate_send() must zero the old outbound key bytearray in-place.
434
435     TAD Reference: Section 3.2 – Memory Safety Rule
436     """
437     state = DKESessionState.from_k0(k0)
438     old_buf = state.outbound.current_key # hold ref to old buffer
439     rotate_send(state, os.urandom(NONCE_SIZE))
440     assert all(b == 0 for b in old_buf), (
441         "Old outbound key buffer must be zeroed after rotation."
442     )
443
444 def test_rotate_send_increment_outbound_seq(self, k0):
445     """rotate_send() must increment outbound.sequence_number."""
446     state = DKESessionState.from_k0(k0)
447     rotate_send(state, os.urandom(NONCE_SIZE))
448     assert state.outbound.sequence_number == 1
449     rotate_send(state, os.urandom(NONCE_SIZE))
450     assert state.outbound.sequence_number == 2
451
452 def test_rotate_receive_advances_inbound_key(self, k0):
453     """rotate_receive() must update inbound.current_key."""
454     state = DKESessionState.from_k0(k0)
455     key_before = bytes(state.inbound.current_key)
456     rotate_receive(state, os.urandom(NONCE_SIZE), received_seq=0)
457     assert bytes(state.inbound.current_key) != key_before
458
459 def test_rotate_receive_does_not_touch_outbound_chain(self, k0):
460     """rotate_receive() must leave outbound.current_key completely unchanged.
461
462     TAD Reference: Section 3.2 – Two Independent Ratchet Chains
463     """
464     state = DKESessionState.from_k0(k0)
465     outbound_key_before = bytes(state.outbound.current_key)
466     outbound_seq_before = state.outbound.sequence_number
467
468     rotate_receive(state, os.urandom(NONCE_SIZE), received_seq=0)
469

```

```

470         assert bytes(state.outbound.current_key) == outbound_key_before
471         assert state.outbound.sequence_number == outbound_seq_before
472
473     def test_rotate_receive_erases_old_inbound_key(self, k0):
474         """rotate_receive() must zero the old inbound key bytearray in-place."""
475         state = DKESessionState.from_k0(k0)
476         old_buf = state.inbound.current_key # hold ref
477         rotate_receive(state, os.urandom(NONCE_SIZE), received_seq=0)
478         assert all(b == 0 for b in old_buf), (
479             "Old inbound key buffer must be zeroed after rotation."
480         )
481
482     def test_rotate_receive_increments_inbound_seq(self, k0):
483         """rotate_receive() must increment inbound.sequence_number."""
484         state = DKESessionState.from_k0(k0)
485         rotate_receive(state, os.urandom(NONCE_SIZE), received_seq=0)
486         assert state.inbound.sequence_number == 1
487
488     def test_rotate_receive_rejects_bad_seq_leaves_state_unchanged(self, k0):
489         """rotate_receive() with a wrong seq must leave ALL state unchanged.
490
491         TAD Reference: Section 7 – Fail-closed on sequence error
492         """
493         state = DKESessionState.from_k0(k0)
494         inbound_key_before = bytes(state.inbound.current_key)
495         inbound_seq_before = state.inbound.sequence_number
496         outbound_key_before = bytes(state.outbound.current_key)
497
498         with pytest.raises(SequenceError):
499             rotate_receive(state, os.urandom(NONCE_SIZE), received_seq=99)
500
501         assert bytes(state.inbound.current_key) == inbound_key_before
502         assert state.inbound.sequence_number == inbound_seq_before
503         assert bytes(state.outbound.current_key) == outbound_key_before
504
505
506     class TestFullSession:
507
508     def test_outbound_and_inbound_chains_are_independent(self, k0):
509         """Advancing the outbound chain must not affect the inbound chain, and v.v.
510
511         TAD Reference: Section 3.2 – Two Independent Ratchet Chains
512         """
513         state = DKESessionState.from_k0(k0)
514         inbound_key_snap = bytes(state.inbound.current_key)
515
516         # Advance outbound 3 times
517         for _ in range(3):
518             rotate_send(state, os.urandom(NONCE_SIZE))
519
520         # Inbound chain must be untouched
521         assert bytes(state.inbound.current_key) == inbound_key_snap
522         assert state.inbound.sequence_number == 0
523
524     def test_cross_match_after_one_message(self, session_states):
525         """sender.outbound.current_key must equal receiver.inbound.current_key
526         immediately after rotate_send / rotate_receive with the same nonce.
527
528         TAD Reference: Section 3.2 – Cross-matching invariant
529         """
530         state_a, state_b = session_states
531
532         # Snapshot Alice's key and seq BEFORE rotation (these are what Bob will use)
533         key_a, seq_a = outbound_params(state_a)
534         nonce = os.urandom(NONCE_SIZE)
535
536         rotate_send(state_a, nonce)

```

```

537
538     # Bob validates seq and retrieves key
539     key_b = inbound_key(state_b, seq_a)
540     assert key_a == key_b, (
541         "Encryption key (Alice outbound) must match decryption key (Bob inbound).")
542     )
543     rotate_receive(state_b, nonce, seq_a)
544
545     # After rotation, Alice's new outbound == Bob's new inbound
546     assert bytes(state_a.outbound.current_key) == bytes(state_b.inbound.current_key), (
547         "Cross-match invariant must hold after rotation."
548     )
549
550 def test_bidirectional_cross_match_stays_consistent(self, session_states):
551     """Both cross-match invariants must hold after 5 interleaved messages.
552
553     Invariants:
554         state_a.outbound.current_key == state_b.inbound.current_key
555         state_b.outbound.current_key == state_a.inbound.current_key
556
557     TAD Reference: Section 6.2 – Steady-State Sequence
558     """
559     state_a, state_b = session_states
560
561     def alice_sends():
562         key_a, seq_a = outbound_params(state_a)
563         n = os.urandom(NONCE_SIZE)
564         rotate_send(state_a, n)
565         key_b = inbound_key(state_b, seq_a)
566         assert key_a == key_b, "Alice-send key mismatch"
567         rotate_receive(state_b, n, seq_a)
568
569     def bob_sends():
570         key_b, seq_b = outbound_params(state_b)
571         n = os.urandom(NONCE_SIZE)
572         rotate_send(state_b, n)
573         key_a = inbound_key(state_a, seq_b)
574         assert key_b == key_a, "Bob-send key mismatch"
575         rotate_receive(state_a, n, seq_b)
576
577     alice_sends()
578     alice_sends()
579     bob_sends()
580     alice_sends()
581     bob_sends()
582
583     # Both cross-match invariants must hold
584     assert bytes(state_a.outbound.current_key) == bytes(state_b.inbound.current_key)
585     assert bytes(state_b.outbound.current_key) == bytes(state_a.inbound.current_key)
586
587 def test_replay_rejected_after_message_consumed(self, session_states):
588     """A replayed envelope (same seq) must be rejected after first delivery.
589
590     TAD Reference: Section 7 – Failure Modes (Replayed message)
591     """
592     state_a, state_b = session_states
593     key_a, seq_a = outbound_params(state_a)
594     nonce = os.urandom(NONCE_SIZE)
595     rotate_send(state_a, nonce)
596
597     # First delivery – OK
598     inbound_key(state_b, seq_a)
599     rotate_receive(state_b, nonce, seq_a)
600
601     # Replay – must be rejected
602     with pytest.raises(SequenceError):
603         inbound_key(state_b, seq_a)

```

```

604
605
606 # =====
607 # protocol tests
608 # =====
609
610
611 class TestEncryptDecrypt:
612
613     def test_encrypt_decrypt_roundtrip(self, aes_key, nonce):
614         """encrypt() followed by decrypt() must recover the original plaintext."""
615         plaintext = b"Hello, DKE!"
616         ct, tag = encrypt(aes_key, plaintext, nonce)
617         assert decrypt(aes_key, ct, nonce, tag) == plaintext
618
619     def test_encrypt_ciphertext_length_equals_plaintext(self, aes_key, nonce):
620         """AES-GCM ciphertext must be the same length as the plaintext."""
621         plaintext = b"A" * 100
622         ct, tag = encrypt(aes_key, plaintext, nonce)
623         assert len(ct) == len(plaintext)
624         assert len(tag) == TAG_SIZE
625
626     def test_encrypt_empty_plaintext(self, aes_key, nonce):
627         """Empty plaintext must encrypt and decrypt cleanly."""
628         ct, tag = encrypt(aes_key, b"", nonce)
629         assert ct == b""
630         assert len(tag) == TAG_SIZE
631         assert decrypt(aes_key, ct, nonce, tag) == b""
632
633     def test_encrypt_different_nonces_yield_different_ciphertexts(self, aes_key):
634         """The same plaintext encrypted under different nonces must differ."""
635         pt = b"same plaintext"
636         ct1, _ = encrypt(aes_key, pt, os.urandom(12))
637         ct2, _ = encrypt(aes_key, pt, os.urandom(12))
638         assert ct1 != ct2
639
640     def test_tampered_ciphertext_raises_on_decrypt(self, aes_key, nonce, encrypted_hello):
641         """Flipping any bit in the ciphertext must cause decrypt() to raise InvalidTag.
642
643         TAD Reference: Section 7.1 – Fail-closed policy
644         """
645         ct, tag = encrypted_hello
646         ct_tampered = bytes([ct[0] ^ 0xFF]) + ct[1:]
647         with pytest.raises(InvalidTag):
648             decrypt(aes_key, ct_tampered, nonce, tag)
649
650     def test_tampered_last_byte_ciphertext_raises(self, aes_key, nonce, encrypted_hello):
651         """Flipping the last byte of ciphertext must also raise InvalidTag."""
652         ct, tag = encrypted_hello
653         ct_tampered = ct[:-1] + bytes([ct[-1] ^ 0xFF])
654         with pytest.raises(InvalidTag):
655             decrypt(aes_key, ct_tampered, nonce, tag)
656
657     def test_tampered_tag_raises_on_decrypt(self, aes_key, nonce, encrypted_hello):
658         """Providing a wrong authentication tag must cause decrypt() to raise InvalidTag.
659
660         TAD Reference: Section 5.2 – Integrity & Authenticity
661         """
662         ct, tag = encrypted_hello
663         tag_tampered = bytes([tag[0] ^ 0xFF]) + tag[1:]
664         with pytest.raises(InvalidTag):
665             decrypt(aes_key, ct, nonce, tag_tampered)
666
667     def test_decrypt_never_returns_on_tag_failure(self, aes_key, nonce, encrypted_hello):
668         """decrypt() must raise, not return None or empty bytes, on tag failure.
669
670         TAD Reference: Section 3.3 – AEAD Contract

```

```

671             Section 7.1 – Fail-closed policy
672         """
673         ct, _ = encrypted_hello
674         tag_bad = bytes(TAG_SIZE) # all-zero tag (almost certainly wrong)
675         result = None
676         try:
677             result = decrypt(aes_key, ct, nonce, tag_bad)
678         except InvalidTag:
679             pass
680         assert result is None
681
682     def test_wrong_key_raises_on_decrypt(self, aes_key, nonce, encrypted_hello):
683         """Decrypting with the wrong key must raise InvalidTag."""
684         ct, tag = encrypted_hello
685         with pytest.raises(InvalidTag):
686             decrypt(os.urandom(KEY_SIZE), ct, nonce, tag)
687
688     def test_encrypt_rejects_wrong_key_size(self, nonce):
689         """encrypt() must raise ValueError for a key that is not 32 bytes."""
690         with pytest.raises(ValueError, match="32 bytes"):
691             encrypt(os.urandom(16), b"hi", nonce)
692
693     def test_decrypt_rejects_wrong_tag_size(self, aes_key, nonce, encrypted_hello):
694         """decrypt() must raise ValueError for a tag that is not 16 bytes."""
695         ct, _ = encrypted_hello
696         with pytest.raises(ValueError, match="16 bytes"):
697             decrypt(aes_key, ct, nonce, os.urandom(8))
698
699
700 class TestEnvelopeFormat:
701
702     @pytest.fixture()
703     def valid_envelope(self, aes_key, nonce):
704         """A correctly encoded envelope with seq=7."""
705         ct, tag = encrypt(aes_key, b"test payload", nonce)
706         envelope = encode_envelope(sequence=7, nonce=nonce, ciphertext=ct, tag=tag)
707         return envelope, ct, tag, nonce
708
709     def test_encode_decode_envelope_roundtrip(self, aes_key, nonce):
710         """encode_envelope() followed by decode_envelope() must recover all fields exactly.
711
712         TAD Reference: Section 4.1 – Envelope Structure
713         """
714         ct, tag = encrypt(aes_key, b"hello", nonce)
715         envelope = encode_envelope(sequence=42, nonce=nonce, ciphertext=ct, tag=tag)
716         seq_out, nonce_out, ct_out, tag_out = decode_envelope(envelope)
717
718         assert seq_out == 42
719         assert nonce_out == nonce
720         assert ct_out == ct
721         assert tag_out == tag
722
723     def test_envelope_minimum_size_constant(self):
724         """MIN_ENVELOPE_SIZE must equal HEADER_SIZE + TAG_SIZE = 46."""
725         assert MIN_ENVELOPE_SIZE == 46
726         assert MIN_ENVELOPE_SIZE == HEADER_SIZE + TAG_SIZE
727
728     def test_encode_envelope_length(self, aes_key, nonce):
729         """Encoded envelope length must equal HEADER_SIZE + len(ct) + TAG_SIZE."""
730         ct, tag = encrypt(aes_key, b"x" * 50, nonce)
731         envelope = encode_envelope(0, nonce, ct, tag)
732         assert len(envelope) == HEADER_SIZE + len(ct) + TAG_SIZE
733
734     def test_encode_sequence_zero(self, aes_key, nonce):
735         """Sequence 0 (first message) must encode and decode correctly."""
736         ct, tag = encrypt(aes_key, b"first", nonce)
737         envelope = encode_envelope(0, nonce, ct, tag)

```

```

738         seq_out, _, _, _ = decode_envelope(envelope)
739         assert seq_out == 0
740
741     def test_encode_large_sequence(self, aes_key, nonce):
742         """Large sequence numbers (uint64 range) must survive encode/decode."""
743         large_seq = (2**63) - 1
744         ct, tag = encrypt(aes_key, b"big seq", nonce)
745         envelope = encode_envelope(large_seq, nonce, ct, tag)
746         seq_out, _, _, _ = decode_envelope(envelope)
747         assert seq_out == large_seq
748
749     def test_decode_rejects_input_shorter_than_minimum(self):
750         """decode_envelope() must raise ParseError for any input < 46 bytes.
751
752         TAD Reference: Section 4.2 – Minimum Envelope Size
753         """
754         with pytest.raises(ParseError, match="too short"):
755             decode_envelope(b"DKE1" + b"\x00" * 10)
756
757     def test_decode_rejects_empty_input(self):
758         """decode_envelope() must raise ParseError for empty input."""
759         with pytest.raises(ParseError):
760             decode_envelope(b"")
761
762     def test_decode_rejects_wrong_magic(self, valid_envelope):
763         """decode_envelope() must raise ParseError when magic != b'DKE1'.
764
765         TAD Reference: Section 7 – Failure Modes (Malformed envelope)
766         """
767         envelope, *_ = valid_envelope
768         bad = bytearray(envelope)
769         bad[0] = ord("X")
770         with pytest.raises(ParseError, match="magic"):
771             decode_envelope(bytes(bad))
772
773     def test_decode_rejects_unsupported_version(self, valid_envelope):
774         """decode_envelope() must raise ParseError for version != 0x01."""
775         envelope, *_ = valid_envelope
776         bad = bytearray(envelope)
777         bad[4] = 0x02
778         with pytest.raises(ParseError, match="version"):
779             decode_envelope(bytes(bad))
780
781     def test_decode_rejects_nonzero_flags(self, valid_envelope):
782         """decode_envelope() must raise ParseError when flags byte != 0x00."""
783         envelope, *_ = valid_envelope
784         bad = bytearray(envelope)
785         bad[5] = 0x01
786         with pytest.raises(ParseError, match="flags"):
787             decode_envelope(bytes(bad))
788
789     def test_decode_rejects_truncated_envelope(self, valid_envelope):
790         """decode_envelope() must raise ParseError when data is shorter than declared."""
791         envelope, *_ = valid_envelope
792         with pytest.raises(ParseError, match="mismatch"):
793             decode_envelope(envelope[:-4])
794
795     def test_decode_rejects_padded_envelope(self, valid_envelope):
796         """decode_envelope() must raise ParseError when data is longer than declared."""
797         envelope, *_ = valid_envelope
798         with pytest.raises(ParseError, match="mismatch"):
799             decode_envelope(envelope + b"\xff\xff")
800
801     def test_decode_rejects_mismatched_ct_len(self, valid_envelope):
802         """decode_envelope() must raise ParseError when ct_len field is inconsistent."""
803         envelope, *_ = valid_envelope
804         bad = bytearray(envelope)

```



```

872     envelope = encode_envelope(seq_a, n, ct, tag)
873     rotate_send(state_a, n)
874
875     # Flip one ciphertext byte inside the envelope (after fixed header)
876     tampered = bytearray(envelope)
877     tampered[HEADER_SIZE] ^= 0xFF
878
879     # Structural parse must succeed
880     seq_r, n_r, ct_r, tag_r = decode_envelope(bytes(tampered))
881
882     # Cryptographic authentication must fail
883     key_b = inbound_key(state_b, seq_r)
884     with pytest.raises(InvalidTag):
885         decrypt(key_b, ct_r, n_r, tag_r)
886
887     def test_session_state_wipe_zeros_all_keys(self, k0):
888         """DKESessionState.wipe() must overwrite both outbound and inbound key buffers with zeros."""
889         state = DKESessionState.from_k0(k0)
890         out_buf = state.outbound.current_key
891         in_buf = state.inbound.current_key
892         assert not all(b == 0 for b in out_buf)
893         assert not all(b == 0 for b in in_buf)
894
895         state.wipe()
896         assert all(b == 0 for b in out_buf)
897         assert all(b == 0 for b in in_buf)
898
899     def test_rotate_receive_rejects_nonce_reuse(self, k0):
900         """rotate_receive() must raise ValueError if the incoming nonce matches the previous message's nonce
901         ,"""
902         state = DKESessionState.from_k0(k0)
903         reused_nonce = os.urandom(NONCE_SIZE)
904         # First receive with reused_nonce succeeds
905         rotate_receive(state, reused_nonce, received_seq=0)
906         # Second receive reusing the exact same nonce must be rejected
907         with pytest.raises(ValueError, match="Nonce reuse detected"):
908             rotate_receive(state, reused_nonce, received_seq=1)
909
910 # =====
911 # Ed25519 Identity Authentication Tests
912 # =====
913
914
915 class TestEd25519Auth:
916     """Tests for Ed25519 identity key generation, signing, and verification.
917
918     Covers the MITM-mitigation layer added in crypto_core.py:
919     - generate_identity_keypair()
920     - sign_public_key()
921     - verify_public_key()
922
923     Test Coverage
924     -----
925     (a) Valid signature allows the session to proceed (no exception raised).
926     (b) Tampered / forged signature causes rejection (InvalidSignature raised).
927     """
928
929     # -----
930     # generate_identity_keypair()
931     # -----
932
933     def test_generate_identity_keypair_returns_correct_types(self):
934         """generate_identity_keypair() returns (private_key_obj, 32-byte public key)."""
935         id_priv, id_pub = generate_identity_keypair()
936         # Public key must be a raw bytes object of exactly IDENTITY_PUBLIC_KEY_SIZE bytes
937         assert isinstance(id_pub, bytes)

```

```

938         assert len(id_pub) == IDENTITY_PUBLIC_KEY_SIZE # 32
939
940     def test_generate_identity_keypair_unique_each_call(self):
941         """Two generate_identity_keypair() calls produce different key pairs."""
942         _, pub1 = generate_identity_keypair()
943         _, pub2 = generate_identity_keypair()
944         assert pub1 != pub2, "Identity public keys must be unique per call"
945
946     # -----
947     # sign_public_key()
948     # -----
949
950     def test_sign_public_key_returns_64_bytes(self):
951         """sign_public_key() returns exactly SIGNATURE_SIZE (64) bytes."""
952         id_priv, _ = generate_identity_keypair()
953         _, ecdh_pub = generate_keypair()
954         sig = sign_public_key(id_priv, ecdh_pub)
955         assert isinstance(sig, bytes)
956         assert len(sig) == SIGNATURE_SIZE # 64
957
958     def test_sign_public_key_rejects_wrong_length_input(self):
959         """sign_public_key() raises ValueError when ecdh_pub_bytes is not 32 bytes."""
960         id_priv, _ = generate_identity_keypair()
961         with pytest.raises(ValueError, match="ecdh_pub_bytes must be exactly 32 bytes"):
962             sign_public_key(id_priv, b"too_short")
963
964     # -----
965     # verify_public_key() – VALID path (criterion a)
966     # -----
967
968     def test_verify_valid_signature_succeeds(self):
969         """(a) A valid signature allows proceed – verify_public_key() returns None."""
970         id_priv, id_pub = generate_identity_keypair()
971         _, ecdh_pub = generate_keypair()
972         sig = sign_public_key(id_priv, ecdh_pub)
973         # Must return None (no exception) for a valid signature
974         result = verify_public_key(id_pub, ecdh_pub, sig)
975         assert result is None
976
977     def test_verify_valid_signature_different_ecdh_keys(self):
978         """(a) Each freshly generated ECDH key verifies with the same identity key."""
979         id_priv, id_pub = generate_identity_keypair()
980         for _ in range(3):
981             _, ecdh_pub = generate_keypair()
982             sig = sign_public_key(id_priv, ecdh_pub)
983             verify_public_key(id_pub, ecdh_pub, sig) # must not raise
984
985     # -----
986     # verify_public_key() – INVALID paths (criterion b)
987     # -----
988
989     def test_verify_rejects_tampered_ecdh_key(self):
990         """(b) Signing key A's ECDH pub, verifying against different ECDH pub → InvalidSignature.
991
992         Models an active MITM that substitutes their own ephemeral key but
993         cannot produce a valid signature for it.
994         """
995         id_priv, id_pub = generate_identity_keypair()
996         _, ecdh_pub_legitimate = generate_keypair()
997         _, ecdh_pub_mitm = generate_keypair() # attacker's substituted key
998
999         sig = sign_public_key(id_priv, ecdh_pub_legitimate)
1000
1001         # Attacker sends their own ECDH key with the legitimate signature
1002         with pytest.raises(InvalidSignature):
1003             verify_public_key(id_pub, ecdh_pub_mitm, sig)
1004

```

```

1005 def test_verify_rejects_forged_signature(self):
1006     """(b) A random 64-byte signature is rejected – InvalidSignature raised."""
1007     id_priv, id_pub = generate_identity_keypair()
1008     _, ecdh_pub = generate_keypair()
1009     forged_sig = os.urandom(SIGNATURE_SIZE) # random garbage, not a valid Ed25519 sig
1010
1011     with pytest.raises(InvalidSignature):
1012         verify_public_key(id_pub, ecdh_pub, forged_sig)
1013
1014 def test_verify_rejects_wrong_identity_key(self):
1015     """(b) Signature made by identity key A, verification with key B → InvalidSignature.
1016
1017     Models the scenario where the verifier has the wrong identity key
1018     (or an attacker tries to use their own identity key).
1019     """
1020     id_priv_a, _ = generate_identity_keypair()
1021     _, id_pub_b = generate_identity_keypair() # a different party's identity key
1022     _, ecdh_pub = generate_keypair()
1023
1024     sig = sign_public_key(id_priv_a, ecdh_pub) # signed by A
1025
1026     with pytest.raises(InvalidSignature):
1027         verify_public_key(id_pub_b, ecdh_pub, sig) # verified against B
1028
1029 def test_verify_rejects_truncated_signature(self):
1030     """(b) A signature shorter than 64 bytes is rejected – ValueError raised."""
1031     id_priv, id_pub = generate_identity_keypair()
1032     _, ecdh_pub = generate_keypair()
1033     # verify_public_key() validates length before calling Ed25519PublicKey.verify()
1034     with pytest.raises(ValueError, match="signature must be exactly 64 bytes"):
1035         verify_public_key(id_pub, ecdh_pub, b"\x00" * 32) # only 32 bytes
1036
1037 def test_verify_rejects_wrong_length_identity_key(self):
1038     """verify_public_key() raises ValueError when identity_pub_bytes is wrong length."""
1039     _, ecdh_pub = generate_keypair()
1040     fake_sig = os.urandom(SIGNATURE_SIZE)
1041     with pytest.raises(ValueError, match="identity_pub_bytes must be exactly 32 bytes"):
1042         verify_public_key(b"tooshort", ecdh_pub, fake_sig)
1043
1044 # -----
1045 # Full handshake integration
1046 # -----
1047
1048 def test_handshake_integration_valid_signatures(self):
1049     """(a) End-to-end: two parties sign/verify each other's ECDH keys → same K0.
1050
1051     Simulates the complete authenticated handshake:
1052     1. Alice and Bob each generate identity and ephemeral ECDH key pairs.
1053     2. Each signs their ECDH public key with their identity private key.
1054     3. Each verifies the other's signature against the known identity key.
1055     4. Both derive the shared secret via ECDH → HKDF → same K0.
1056     """
1057     # Identity keys (long-term, known out-of-band)
1058     id_priv_alice, id_pub_alice = generate_identity_keypair()
1059     id_priv_bob, id_pub_bob = generate_identity_keypair()
1060
1061     # Ephemeral ECDH keys (per-session)
1062     priv_alice, pub_alice = generate_keypair()
1063     priv_bob, pub_bob = generate_keypair()
1064
1065     # Sign ECDH public keys
1066     sig_alice = sign_public_key(id_priv_alice, pub_alice)
1067     sig_bob = sign_public_key(id_priv_bob, pub_bob)
1068
1069     # Each party verifies the other – must not raise
1070     verify_public_key(id_pub_alice, pub_alice, sig_alice) # Bob verifies Alice
1071     verify_public_key(id_pub_bob, pub_bob, sig_bob) # Alice verifies Bob

```

```
1072
1073     # ECDH → HKDF → K0
1074     secret_alice = derive_shared_secret(priv_alice, pub_bob)
1075     secret_bob = derive_shared_secret(priv_bob, pub_alice)
1076     assert secret_alice == secret_bob, "ECDH shared secrets must match"
1077
1078     k0_alice = hkdf_derive(secret_alice, info=SESSION_INFO)
1079     k0_bob = hkdf_derive(secret_bob, info=SESSION_INFO)
1080     assert k0_alice == k0_bob, "Derived K0 must be identical on both sides"
1081     assert len(k0_alice) == 32
1082
1083 def test_handshake_integration_mitm_tampered_sig_rejected(self):
1084     """(b) MITM substitutes an ephemeral key → InvalidSignature before HKDF.
1085
1086     An active MITM replaces Alice's genuine ECDH public key (pub_alice) with
1087     their own (pub_mitm) while forwarding the original signature. Bob
1088     detects the substitution during verify_public_key() and MUST raise
1089     InvalidSignature before any key derivation takes place.
1090     """
1091     id_priv_alice, id_pub_alice = generate_identity_keypair()
1092
1093     # Alice generates her ephemeral key and signs it
1094     priv_alice, pub_alice = generate_keypair()
1095     sig_alice = sign_public_key(id_priv_alice, pub_alice)
1096
1097     # MITM substitutes a different ECDH key while reusing Alice's signature
1098     _, pub_mitm = generate_keypair()
1099
1100     # Bob verifies: the substituted key MUST be rejected
1101     with pytest.raises(InvalidSignature):
1102         verify_public_key(id_pub_alice, pub_mitm, sig_alice)
```